

Dion3: Full-stack orthogonal updates

Noah Amsel^{*1}, Jack Zhang², Kwangjun Ahn^{*3}, Ali Naeimi⁴, Austin Feng^{*5}, Berlin Chen², Tri Dao², and John Langford⁶

¹New York University, ²Princeton University, ³NVIDIA, ⁴Independent Researcher, ⁵Yale University,

⁶Microsoft Research

August 13, 2026

Abstract

The Muon optimizer incurs a significant overhead cost due to its cubic-time Newton-Schulz orthogonalization step. When weights are sharded, communication overhead compounds this computational cost, eroding the benefits of Muon in many settings. We present Dion3, a revision of Muon that targets this overhead at every level of the stack. Our Gram Newton-Schulz algorithm reduces the FLOP cost of orthogonalization, our CuteDSL kernels accelerate it by exploiting symmetry, and our megabatching strategy reduces communication overhead. Moreover, we propose a simple change to the update rule that cuts costs even further: selecting only a fraction of the momentum matrix’s rows to orthogonalize at each step. This update rule improves on Dion (another “compressed” version of Muon), in both speed and performance [2]. Overall, Dion3 matches or improves on the loss achieved by Muon but reduces optimizer step time by up to 6×. Dion3 is available via the `dion` package¹ as a drop-in replacement for Muon.

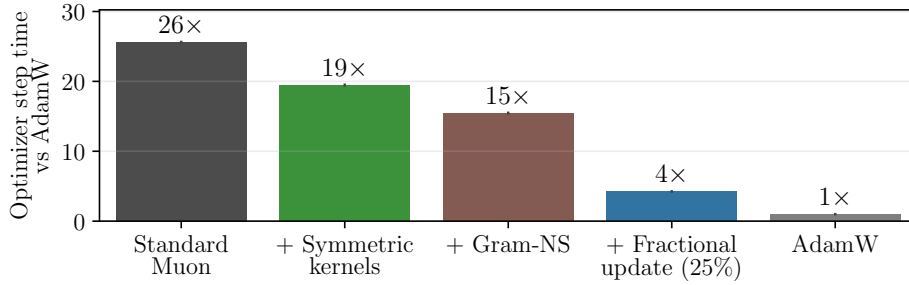


Figure 1: Optimizer step time of Muon (excluding forward/backward pass) relative to AdamW for a 7B-parameter language model trained on four GH200s. Each colored bar adds one of our contributions on top of the previous one. Together they reduce Muon’s cost from 26× AdamW to just 4×.

^{*}Work performed while at Microsoft Research.

¹<https://github.com/microsoft/dion>

Contents

1	Introduction	3
2	The Challenge of Scaling Muon	3
2.1	Muon and NorMuon Recap	3
2.2	Standard Newton-Schulz	4
2.3	Scalability of Muon	5
3	Comparison with Related Work	6
3.1	Improving Newton-Schulz	6
3.2	Orthogonalizing a Smaller Matrix	7
4	Gram Newton-Schulz	7
4.1	Runtime of Naive Gram Newton-Schulz	8
4.2	Stabilizing Gram Newton-Schulz	9
5	Symmetric GEMM Kernels in CuteDSL	10
6	The Dion3 Update Rule	11
7	Megabatching and Communication	13
8	Experiments	14
8.1	Model Quality Is Preserved	14
8.2	Dion3 Accelerates the Optimizer	15
9	Conclusion	16
	References	17
	Appendices	20
A	Alternative Experiments on Gram Newton-Schulz	20
A.1	Setup	20
A.2	Kernelized Gram Newton-Schulz preserves quality	21
A.3	Kernelized Gram Newton-Schulz speeds up the optimizer step	21
B	Stability of Gram Newton-Schulz	23
B.1	Instability of Naive Gram Newton-Schulz	23
B.2	Stabilizing Gram Newton-Schulz by Restarting	27
C	Kernel Implementation Details	31
C.1	Symmetric GEMM Kernel Details	31
C.2	Implementation Strategy in Code	31
C.3	Kernel Optimizations for Standard Newton-Schulz	32
D	Additional Experiments	32
D.1	Architecture and Optimization Setup	32
D.2	Finer-Grained Timing Metrics	33
D.3	Benchmarking all-to-all communications	33
D.4	Ablations	34
E	Case Studies of End-to-End Training Time	35

1 Introduction

Muon is becoming the method of choice for training frontier LLMs like Kimi K2 and GLM-5 [13, 21]. Compared to AdamW, Muon needs fewer optimizer steps to reach a given loss, but each step is more expensive. This overhead is due to Muon’s Newton-Schulz orthogonalization procedure, a cubic-time matrix operation not present in older optimizers. As model size increases, the overhead of computing each Muon step grows rapidly. Moreover, Muon is more difficult than traditional optimizers to parallelize, introducing additional communication overhead in distributed settings. Thus, scaling Muon presents a range of algorithmic and systems-level challenges that diminish its effectiveness in the most demanding settings and hinder its adoption to new problems.

This paper gives a comprehensive answer to these challenges. We provide a full-stack solution that performs well across a wide range of model sizes, architectures, cluster sizes, and parallelism strategies. We package our contributions into a single optimizer, Dion3, built on four improvements that each reduce the cost of Muon’s orthogonalization step and compound when used together:

1. **Gram Newton-Schulz**, a mathematically equivalent reformulation of Newton-Schulz that iterates on the small symmetric Gram matrix and cuts the FLOP cost of each orthogonalization dramatically (Section 4).
2. Custom **GPU kernels for symmetric matrix multiplication** written in CuteDSL, which speed up Gram Newton-Schulz and take maximal advantage of its symmetric structure (Section 5).
3. A new **optimizer update rule** that subsamples the rows or columns of the momentum matrix before orthogonalizing it to make each step faster (Section 6). Our update rule is simpler and faster than Dion [2] while matching or improving the optimization quality of Muon and NorMuon.
4. **Megabatched communication**, which eliminates a significant source of overhead by reducing the number of rounds of communication per optimizer step to a small constant.

Our improvements combine to accelerate the optimizer step at every level of the stack. The GPU kernels at the lowest level speed up Gram Newton-Schulz, which in turn reduces the FLOP cost of executing our optimizer’s update rule. At the top level, the compression achieved by our update rule joins with the megabatch strategy to limit the communication cost. Each of our contributions is of independent value, but combining them gives Dion3 maximum flexibility to handle different parallelism strategies, architectures, and model sizes efficiently. Moreover, Dion3 benefits from synergies between them that further reduce computational cost. Gram Newton-Schulz uses more symmetric multiplications than standard Newton-Schulz does, compounding the benefits of our CuteDSL kernels. Likewise, Gram Newton-Schulz is especially fast for matrices with highly asymmetric shapes—like those produced by our update rule’s subsampling strategy. Surprisingly, Dion3 appears to benefit the update *quality* as well as the computational footprint in some settings, even with a strong optimized baseline.

To realize these gains in practice, we provide open-source implementations in the form of two interoperable packages: `gram-newton-schulz`² is a drop-in replacement for Muon’s Newton-Schulz routine that also incorporates our CuteDSL kernels, while the `dion` package implements our optimizer update rule (along with baselines like Muon and NorMuon), and handles parallelism in the distributed setting (megabatching, FSDP, DDP, etc.). In all, Dion3 allows practitioners working in a wide range of settings to realize the benefits of Muon and related optimizers with almost no overhead.

2 The Challenge of Scaling Muon

2.1 Muon and NorMuon Recap

The Muon optimizer [19] is best described as steepest-direction descent with respect to the spectral norm [5]. At a given training step, let $\mathbf{W} \in \mathbb{R}^{n \times m}$ be a weight matrix and let $\mathbf{G}$ be the gradient of the loss with respect to $\mathbf{W}$. The Muon update rule is

$$\begin{aligned}\mathbf{M} &\leftarrow \mu \mathbf{M} + \mathbf{G} \\ \mathbf{W} &\leftarrow \mathbf{W} - \eta \text{polar}(\mathbf{M})\end{aligned}\tag{1}$$

²<https://github.com/Dao-AILab/gram-newton-schulz>

where μ is the momentum coefficient, η is the learning rate, and $\mathbf{M}$ is the momentum matrix (with $\mathbf{M}_0 := \mathbf{0}$). In many ways, Muon resembles basic stochastic gradient descent (SGD) with momentum. Its key innovation is the polar operation, which is defined as follows:

Definition 1 (Polar Decomposition). *If $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the singular value decomposition (SVD) of a matrix, then $\text{polar}(\mathbf{X}) = \mathbf{U}\mathbf{V}^\top$.*

This operation—the “orthogonalization step”—balances the spectrum of the update, ensuring that it has full numerical rank.

NorMuon [23] is a popular variant of Muon that aims to combine its strengths with those of Adam. While Muon’s update has a balanced spectrum, it does not have balanced row norms; each step could greatly affect some neurons while almost neglecting others. Following Adam, NorMuon uses the second moment of the (orthogonalized) gradients to adaptively adjust the stepsize for each neuron, helping all neurons to learn at every step. A final rescaling ensures that the overall magnitude of the update (as measured in the Frobenius norm) matches that of Muon:

$$\begin{aligned} \mathbf{M} &\leftarrow \mu\mathbf{M} + \mathbf{G}, & \mathbf{O} &\leftarrow \text{polar}(\mathbf{M}) \\ \mathbf{v}_i &\leftarrow \beta_2\mathbf{v}_i + (1 - \beta_2) \cdot \frac{1}{m} \sum_j \mathbf{o}_{ij}^2, & \hat{\mathbf{o}}_{ij} &\leftarrow \frac{\mathbf{o}_{ij}}{\sqrt{\mathbf{v}_i} + \epsilon} \\ \mathbf{W} &\leftarrow \mathbf{W} - \eta \frac{\|\mathbf{O}\|_F}{\|\hat{\mathbf{O}}\|_F} \hat{\mathbf{O}} \end{aligned} \tag{2}$$

The cost of NorMuon’s extra normalization steps is negligible compared to that of computing $\text{polar}(\mathbf{M})$, so NorMuon’s benefits come almost for free. Our methods, which speed up the computation of $\text{polar}(\mathbf{M})$, apply to both Muon and NorMuon alike.

2.2 Standard Newton-Schulz

The main challenge to scaling Muon is the need to compute $\text{polar}(\mathbf{M})$ for each weight matrix at every step. Since $\text{polar}(\cdot)$ is expensive to compute exactly, Muon uses the Newton-Schulz method to approximate it. Newton-Schulz is an iterative method based on matrix polynomials. Beginning with $\mathbf{X}_0$, each iteration improves the approximation $\mathbf{X}_t \approx \text{polar}(\mathbf{X}_0)$ according to the update rule

$$\mathbf{X}_{t+1} = a_t\mathbf{X}_t + b_t\mathbf{X}_t\mathbf{X}_t^\top\mathbf{X}_t + c_t(\mathbf{X}_t\mathbf{X}_t^\top)^2\mathbf{X}_t.$$

We can interpret Newton-Schulz by understanding how it affects the singular value decomposition. Let $\mathbf{X}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ be the SVD, where $\mathbf{U}^\top\mathbf{U} = \mathbf{V}^\top\mathbf{V} = \mathbf{I}$ and $\mathbf{\Sigma}$ is diagonal with positive entries called the singular values. A direct computation shows

$$\mathbf{X}_1 = \mathbf{U}(a_1\mathbf{\Sigma} + b_1\mathbf{\Sigma}^3 + c_1\mathbf{\Sigma}^5)\mathbf{V}^\top = \mathbf{U}p_1(\mathbf{\Sigma})\mathbf{V}^\top$$

where $p_1(x) := a_1x + b_1x^3 + c_1x^5$. Since $\mathbf{U}$ and $\mathbf{V}$ have orthonormal columns and $p_1(\mathbf{\Sigma})$ is diagonal, the right-hand side of this equation must be the SVD of $\mathbf{X}_1$. By extension, $\mathbf{X}_T$ also has the same singular vectors $\mathbf{U}$ and $\mathbf{V}$ as $\mathbf{X}_0$, and its singular values have been transformed according to the composition of polynomials $(p_T \circ \dots \circ p_1)(\mathbf{\Sigma})$. If we normalize the input matrix $\mathbf{X}_0 = \mathbf{X}/\|\mathbf{X}\|_F$, then all singular values of $\mathbf{X}_0$ lie in $[0, 1]$. Jordan et al. [19] identified a sequence of degree-5 odd polynomials for which $(p_T \circ \dots \circ p_1)(x) \approx 1$ on $[0, 1]$. Therefore,

$$\mathbf{X}_T = \mathbf{U}(p_T \circ \dots \circ p_1)(\mathbf{\Sigma})\mathbf{V}^\top \approx \mathbf{U}\mathbf{V}^\top =: \text{polar}(\mathbf{X}_0)$$

Algorithm 1 Standard Newton-Schulz

Input: $\mathbf{X} \in \mathbb{R}^{n \times m}$ with $n \leq m$, coefficients $\{(a_t, b_t, c_t)\}_{t=1}^5$

- 1: $\mathbf{X} \leftarrow \mathbf{X} / (\|\mathbf{X}\|_F + \epsilon)$ ▷ Normalize sing. vals. to $[0, 1]$; $\epsilon = 10^{-7}$
 - 2: $\mathbf{X} \leftarrow \text{bfloat16}(\mathbf{X})$ ▷ Cast to half precision for speed
 - 3: **for** $t = 1, \dots, 5$ **do** ▷ Apply $p_t(\mathbf{X})$
 - 4: $\mathbf{A} \leftarrow \mathbf{X}\mathbf{X}^\top$
 - 5: $\mathbf{B} \leftarrow b_t\mathbf{A} + c_t\mathbf{A}^2$
 - 6: $\mathbf{X} \leftarrow a_t\mathbf{X} + \mathbf{B}\mathbf{X}$
 - 7: **end for**
 - 8: **return** $\mathbf{X}$
-

Algorithm 1 gives the standard implementation of Newton-Schulz. We now analyze its runtime in FLOPs to help us understand its performance bottlenecks. We count only the cubic-time matrix multiplication operations, ignoring the lower-order scalar multiplications and matrix additions. For clarity, we let T denote the number of iterations, remembering that Muon typically sets $T = 5$. We also assume without loss of generality that $n \leq m$ and define the aspect ratio $\alpha = m/n \geq 1$. Intuitively, α measures how asymmetric the shape of the matrix is, with $\alpha = 1$ being square and $\alpha \gg 1$ being very asymmetric.

Each iteration has three steps. Each step contains a single matrix multiplication ($\mathbf{X}\mathbf{X}^\top$, $\mathbf{A}^2$, $\mathbf{B}\mathbf{X}$) costing, respectively, $2mn^2$, $2n^3$, and $2mn^2$ FLOPs for a total cost of $T(4mn^2 + 2n^3) = 2T(2\alpha + 1)n^3$ FLOPs. When $T = 5$, the cost is $(20\alpha + 10)n^3$ spread across 15 matrix-matrix multiplications (GEMMs).

2.3 Scalability of Muon

While Muon comfortably outperforms traditional optimizers like Adam on a per-step basis, two factors immediately make it more difficult to scale:

- **Super-linear complexity.** SGD and Adam perform only inexpensive element-wise operations like addition and scalar multiplication; therefore, the computational complexity of these methods scales linearly with the number of parameters. Orthogonalization is substantially more expensive; for an $n \times n$ weight matrix, it requires $O(n^3)$ time. Although sparse MoE architectures keep most weight matrices smaller, they also reduce overall model FLOPs, increasing the relative cost of the optimizer step [11] compared to the forward and backward passes. Moreover, MoEs still include large weight matrices in their dense layers [9].
- **Distributed training.** There are additional challenges when weights are sharded across GPUs, as is standard in distributed training. Element-wise optimizers like Adam can update each shard separately, but Muon must gather the shards together to compute polar on the full matrix. Early implementations of Muon duplicated the orthogonalization step across devices, but this increases its already considerable cost. Essential AI [11] propose using all-to-all communication along the sharding dimension, with each device processing a different weight in parallel. Ahn et al. [2] implemented this strategy in PyTorch FSDP2, and Lim et al. [24] examined its compute-communication overlap characteristics. While this approach makes the overhead of distributed Newton-Schulz manageable in some settings, it does not fully resolve the scalability limitations.

In Section 6, we introduce a variant of Muon that addresses these two challenges by subselecting (that is, compressing) the momentum matrix before orthogonalizing it. Section 7 describes a megabatching strategy and a flexible, pip-installable package that gracefully handle the distributed setting. Our other contributions are inspired by the analysis in the previous section, which reveals two shortcomings of the standard Newton-Schulz algorithm:

- **Symmetric Matrix Multiplication.** The matrices $\mathbf{A} = \mathbf{X}\mathbf{X}^\top$ and $\mathbf{B} = b_t\mathbf{A} + c_t\mathbf{A}^2$ computed at each iteration of Newton-Schulz are symmetric by definition, but standard Newton-Schulz does not exploit this structure. We can compute the lower triangular part of these matrices in the usual way and simply copy the results to the upper triangular part. This technique halves the cost of computing $\mathbf{X}\mathbf{X}^\top$ and $\mathbf{A}^2$, giving an overall total of $T(3\alpha + 1)n^3$ FLOPs. Section 5 describes custom CuteDSL kernels that implement this technique.
- **Dependence on Aspect Ratio.** Newton-Schulz’s runtime is dominated by the large rectangular matrix multiplications needed to compute $\mathbf{X}\mathbf{X}^\top$ and $\mathbf{B}\mathbf{X}$, which together cost $3\alpha n^3$ FLOPs per iteration even when using symmetric matrix multiplications. A typical implementation with $T = 5$ requires 10 of these expensive rectangular multiplications. This strong dependence on α is unfortunate. Most of the weight matrices in transformer architectures are rectangular, including the MLP weights, MoE weights, and attention projection weights when using GQA or MLA. Furthermore, we observe that the latest MoE architectures are trending towards finer-grained, sparser experts, meaning that the aspect ratios of their hidden dimensions to intermediate dimensions are increasing as well [16, 21, 31, 34]. Thus, at large scales, pretraining time would benefit greatly from an algorithm that uses fewer rectangular multiplications and more small symmetric ones. We develop such an algorithm in Section 4.

2.3.1 Kimi’s Success

Muon has been scaled successfully by Moonshot AI [21, 27]. Their success was enabled by an alignment of several factors:

1. Earlier releases of PyTorch and Megatron-LM used DP-sharding strategies for optimizer states that were, by chance, favorable for Muon [26]. Model and optimizer states were stored in large contiguous flat buffers, and data-parallel shards were produced by splitting this buffer. As a result, only tensors crossing a DP boundary required an additional gather. These advantageous strategies, however, have been deprecated in more recent releases.
2. They adopt a fine-grained MoE architecture with only a single dense layer (even fewer than in DeepSeek-V3 [9]), so most matrices remain small even at the one-trillion-parameter scale, keeping the Newton-Schulz overhead manageable.
3. Their main distributed training strategy combines pipeline parallelism with expert parallelism, which naturally distributes Muon’s computation across devices with minimal communication.

In sum, Muon has been successfully scaled, but its success relies on a subtle alignment of architectural, parallelism, and framework factors. For Muon to serve as a general-purpose replacement for Adam, it would benefit from cheaper, more flexible scaling properties that relax these constraints. This is the goal of the present work.

3 Comparison with Related Work

As Muon has gained popularity, successive work has sought to improve it in several ways. Most of these proposals (e.g. NorMuon) modify Muon’s update rule so as to reach a given loss in fewer training steps; however, they use the same Newton-Schulz routine described above and generally suffer from the same scaling challenges.

3.1 Improving Newton-Schulz

A few papers *have* attempted to improve the Newton-Schulz step itself—e.g., by optimizing the sequence of polynomials (a_t, b_t, c_t) or the normalization step [3, 6, 15]—but they retain the form of Algorithm 1. In contrast, our Gram Newton-Schulz algorithm departs from this form. Since its output is still mathematically identical to the standard version, it remains compatible with nearly all varieties of Muon, including prior improvements to Newton-Schulz.

Gram Newton-Schulz is closely akin to a method proposed in Appendix J of Amsel et al. [3]. Both aim to reduce the FLOP cost of Newton-Schulz, both form the Gram matrix to reduce the number of $m \times n$ matrix multiplications, and both are mathematically identical to standard Newton-Schulz. However, our work supersedes Amsel et al. [3] in several ways. First, the precise formulas of Gram Newton-Schulz are different and, we believe, more stable. Second, we use symmetric matrix multiplication kernels; the opportunity to use these kernels more is an essential advantage of Gram Newton-Schulz not studied previously. Third, we undertake a thorough stability analysis and provide practical recommendations that allow Gram Newton-Schulz to be used in practice with minimal ad-hoc hyperparameter tuning. The iterative part of the method, which approximates the inverse square root of the Gram matrix $\mathbf{Q}_T \approx \mathbf{R}_0^{-1/2} := (\mathbf{X}\mathbf{X}^\top)^{-1/2}$, generalizes and speeds up the method of Lakić [22] (see also Higham [17, Eq. 7.18]), but Lakić does not consider the symmetric kernels, the stability issues, or the application to $\text{polar}(\mathbf{X})$.

The idea of exploiting symmetry to reduce the arithmetic cost of computing $\mathbf{A}^\top \mathbf{A}$ has appeared before, both in standard linear algebra packages like BLAS and in the context of Muon’s Newton-Schulz routine [25, 30, 33]. However, our results show that the true potential of this trick is only realized when combined with Gram Newton-Schulz, which requires more general symmetric operations like $\mathbf{A}\mathbf{B} + \beta\mathbf{C}$. Furthermore, our symmetric matrix multiplication kernels are written in CuteDSL, exploiting the advanced features of NVIDIA’s Hopper and Blackwell GPUs.

3.2 Orthogonalizing a Smaller Matrix

Our update rule is part of a second line of work that reduces the cost of the Newton-Schulz step by shrinking the size of its input, an approach pioneered by Dion [2]. Dion constructs a low-rank approximation of the momentum matrix and orthogonalizes only this approximation. If $\mathbf{M} \approx \mathbf{M}\hat{\mathbf{V}} \cdot \hat{\mathbf{V}}^\top$ is a rank- k approximation with $\hat{\mathbf{V}}^\top \hat{\mathbf{V}} = \mathbf{I} \in \mathbb{R}^{k \times k}$, then

$$\text{polar}(\mathbf{M}) \approx \text{polar}(\mathbf{M}\hat{\mathbf{V}}\hat{\mathbf{V}}^\top) = \text{polar}(\mathbf{M}\hat{\mathbf{V}})\hat{\mathbf{V}}^\top. \quad (3)$$

Because the dimensions of $\mathbf{M}\hat{\mathbf{V}}$ are much smaller than those of $\mathbf{M}$, this approximation greatly reduces the runtime of Newton-Schulz. In Dion, $\hat{\mathbf{V}}$ is found by a warm-started power-iteration procedure. If $\hat{\mathbf{V}}$ spans the top- k right singular vectors of $\mathbf{M}$, then this approximation gives the optimal rank- k approximation of $\text{polar}(\mathbf{M})$. Ahn et al. [2] found that approximating $\hat{\mathbf{V}}$ from only a single warm-started power-iteration suffices for good downstream performance. A key element of Dion’s success is *error feedback*, a technique that helps offset the error introduced by low-rank approximation in future iterations. We adopt error feedback and describe it fully in Section 6.

TRION [29] adopts the main ideas of DION, but changes how the low-rank approximation is computed. Instead of constructing $\hat{\mathbf{V}}$ via power iteration, it selects k columns from the discrete cosine transform matrix. Surprisingly, this simpler approximation performs even better than DION, suggesting that finding a *good* low-rank approximation is not essential; the error-feedback mechanism compensates even for large differences between $\mathbf{M}$ and $\mathbf{M}\hat{\mathbf{V}}\hat{\mathbf{V}}^\top$.

We push this simplification further by using the simplest possible “low-rank approximation” of $\mathbf{M}$: we select k of its rows or columns and do not operate on the others. This achieves the same savings in the runtime of Newton-Schulz as Dion and Trion, but makes all the other steps of the algorithm (construction of $\mathbf{V}$, error feedback, etc.) much easier to implement, especially in the distributed setting.

Besides low-rank approximation, block orthogonalization is an alternative way to shrink the size of the input to Newton-Schulz. This approach partitions the momentum matrix into blocks—each typically corresponding to a shard in weight-sharded settings—and orthogonalizes each block separately. Several papers have proposed versions of this approach [7, 20, 32]. In particular, Khaled et al. [20] found success by periodically alternating block-wise steps with full steps, a method they call MUONBP.

Both low-rank and block orthogonalization methods use Newton-Schulz as a black box. Therefore, they are trivial to combine with improvements to Newton-Schulz.

4 Gram Newton-Schulz

Our first contribution is an orthogonalization algorithm that uses fewer rectangular matrix multiplications than standard Newton-Schulz. Instead of iterating directly on the large input matrix $\mathbf{X}$, we iterate on the small symmetric Gram matrix $\mathbf{X}\mathbf{X}^\top$. The output of our algorithm is mathematically identical to that of standard Newton-Schulz, but it is significantly cheaper to compute.

At a high level, our strategy is based on the following formula. If $\mathbf{X} \in \mathbb{R}^{n \times m}$ with $n \leq m$, then $\text{polar}(\mathbf{X}) = (\mathbf{X}\mathbf{X}^\top)^{-1/2}\mathbf{X}$. Rather than use an iterative method to approximate $\mathbf{X}_T \approx \text{polar}(\mathbf{X})$ directly, we instead:

1. Compute the $n \times n$ Gram matrix $\mathbf{X}\mathbf{X}^\top$
2. Use an iterative method to approximate $\mathbf{Q}_T \approx (\mathbf{X}\mathbf{X}^\top)^{-1/2}$
3. Output $\mathbf{Q}_T\mathbf{X}$

Step 2—which comprises almost all of the algorithm’s wall clock runtime and FLOP cost—works entirely with small $n \times n$ symmetric matrices. This version uses just two rectangular matrix multiplications: $\mathbf{X}\mathbf{X}^\top$ in the beginning and $\mathbf{Q}_T\mathbf{X}$ at the end. It also synergizes well with our symmetric GEMM kernels (Section 5). Because we use more symmetric multiplications than before, these kernels provide an even greater speedup. Since our algorithm works on the $n \times n$ Gram matrix of $\mathbf{X}$, we call it “Gram Newton-Schulz”.

How can we ensure that the output matches that of standard Newton-Schulz? Recall that Newton-Schulz outputs $(p_T \circ \dots \circ p_1)(\mathbf{X})$, where each p_t is an odd polynomial $p(x) = ax + bx^3 + cx^5$. Any odd polynomial

can be rewritten in the form $p(x) = xh(x^2)$, where h is a lower-degree polynomial with the same coefficients, like $h(x) = a + bx + cx^2$. Intuitively, if $p(x) \approx 1$, then $h(y) = p(y^{1/2})y^{-1/2} \approx y^{-1/2}$, so the Newton-Schulz polynomials implicitly provide a way to approximate inverse square roots. If we use this approximation in step 2 of Gram Newton-Schulz, then its output will match that of standard Newton-Schulz.

Formally, Gram Newton-Schulz is based on the following theorem. In effect, it shows how to compute $\mathbf{X}_T$ from $\mathbf{X}_0$ without ever constructing the intermediate values $\mathbf{X}_1, \dots, \mathbf{X}_{T-1}$:

Theorem 2. *If $p_t(x) = xh_t(x^2)$ for all $t \in \{1, \dots, T\}$, then $(p_T \circ \dots \circ p_1)(x) = q_T x$, where q_T is defined by the iteration $r_0 = x^2$, $q_0 = 1$, and*

$$z_t = h_t(r_{t-1}), \quad r_t = r_{t-1}z_t^2, \quad q_t = q_{t-1}z_t$$

for all $t \in \{1, \dots, T\}$.

Proof. Define $x_0 = x$ and $x_t = p_t(x_{t-1})$ for $t \in \{1, \dots, T\}$. We will show by induction that $r_t = x_t^2$ and $q_t = x_t/x_0$ for all t . The base case $t = 0$ holds by the definition $r_0 = x^2, q_0 = 1$. Now assume the hypothesis holds for $t - 1$. By assumption,

$$x_t = p_t(x_{t-1}) = x_{t-1}h_t(x_{t-1}^2)$$

By the inductive hypothesis, $h_t(x_{t-1}^2) = h_t(r_{t-1}) = z_t$, so $x_t = x_{t-1}z_t$. Squaring both sides,

$$x_t^2 = x_{t-1}^2 z_t^2 = r_{t-1} z_t^2 = r_t$$

If we instead divide both sides by x_0 and apply the other part of the inductive hypothesis, we get

$$\frac{x_t}{x_0} = \frac{x_{t-1}}{x_0} z_t = q_{t-1} z_t = q_t$$

Thus, both parts of the hypothesis hold for t . Finally, conclude $(p_T \circ \dots \circ p_1)(x) = x_T = q_T x_0$. $\square$

Note that, as an immediate corollary of the proof, $q_t = x_t/x_0 \rightarrow 1/x_0 = (x_0^2)^{-1/2}$. In effect, this shows that $\mathbf{Q}_T \rightarrow (\mathbf{X}\mathbf{X}^\top)^{-1/2}$.

To obtain our initial version of Gram Newton-Schulz, we simply lift the iteration from [Theorem 2](#) to matrices. As in standard Newton-Schulz, each matrix operation preserves singular vectors. Therefore, each singular value of $\mathbf{R}_t$, $\mathbf{Q}_t$, and $\mathbf{Z}_t$ evolves independently of the others according to the scalar iteration described above. Note that while this algorithm is mathematically equivalent to standard Newton-Schulz, it is not yet practical due to numerical instability. The only difference between our proposed method ([Algorithm 3](#)) and this naive version is the presence of what we call a “restart” at the beginning of iteration 3 of the loop. We will motivate this change below ([Section 4.2](#)).

Algorithm 2 Naive Gram Newton-Schulz

Input: $\mathbf{X} \in \mathbb{R}^{n \times m}$ with $n \leq m$, coefficients $\{(a_t, b_t, c_t)\}_{t=1}^5$

- 1: $\mathbf{X} \leftarrow \mathbf{X} / (\|\mathbf{X}\|_F + \epsilon)$ $\triangleright$ Normalize sing vals to $[0, 1]$, $\epsilon = 10^{-7}$
 - 2: $\mathbf{R}_0 = \mathbf{X}\mathbf{X}^\top$
 - 3: $\mathbf{Q}_0 = \mathbf{I}$
 - 4: **for** $t = 1, \dots, 5$ **do**
 - 5: $\mathbf{Z}_t \leftarrow a_t \mathbf{I} + b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$ $\triangleright$ Apply $h_t(\mathbf{R}_{t-1})$
 - 6: $\mathbf{Q}_t \leftarrow \mathbf{Q}_{t-1} \mathbf{Z}_t$
 - 7: $\mathbf{R}_t \leftarrow \mathbf{Z}_t \mathbf{R}_{t-1} \mathbf{Z}_t$
 - 8: **end for**
 - 9: **return** $\mathbf{Q}_5 \mathbf{X}$
-

4.1 Runtime of Naive Gram Newton-Schulz

We now calculate the FLOP count of this new algorithm to show how its runtime improves on standard Newton-Schulz. There are four matrix multiplications per iteration; if we use our symmetric GEMM kernel, these cost n^3 FLOPs each. The initialization $(\mathbf{X}\mathbf{X}^\top)$ and output $(\mathbf{Q}_5 \mathbf{X})$ steps cost mn^2 and $2mn^2$, respectively, since $\mathbf{Q}_5 \mathbf{X}$ is not symmetric. Computing $\mathbf{Q}_1 = \mathbf{Q}_0 \mathbf{Z}_1$ is free since $\mathbf{Q}_0 = \mathbf{I}$, and we can skip computing

$\mathbf{R}_5 = \mathbf{Z}_5 \mathbf{R}_4 \mathbf{Z}_5$; together this saves $3n^3$ FLOPs. Thus, the total cost is $T \cdot 4n^3 + 3mn^2 - 3n^3 = (4T + 3\alpha - 3)n^3$ FLOPs.

Compare this to standard Newton-Schulz’s $T(3\alpha + 1)n^3$ FLOPs when using symmetric GEMMs. When $\alpha = 1$, they are equal. When $\alpha > 1$, Gram Newton-Schulz is cheaper, often significantly so. For a typical Muon application ($T = 5, \alpha = 4^3$), it saves **55%** of the FLOPs used by standard Newton-Schulz with symmetric GEMMs, or **68%** compared to a typical implementation without symmetric GEMMs. For larger T and α , the savings are even greater, as the leading order term is $O((T + \alpha)n^3)$ instead of $O(T\alpha n^3)$.

In practice, when $\alpha = 1$, we fall back to standard Newton-Schulz with our symmetric GEMMs (Appendix C.3), since it launches fewer GEMMs and has a faster wall clock time.

4.2 Stabilizing Gram Newton-Schulz

As written, Algorithm 2 destabilizes Muon and can even diverge. The leading cause of this instability is the introduction of spurious negative eigenvalues in the Gram matrix $\mathbf{X}\mathbf{X}^\top$ due to large rounding errors in half-precision arithmetic. These negative eigenvalues theoretically should not exist in a Gram matrix like $\mathbf{X}\mathbf{X}^\top$. The main loop of Gram Newton-Schulz approximates inverse square root of positive numbers, but it diverges for negative inputs. We provide a thorough theoretical analysis and numerical experiments showing the impact of spurious negative eigenvalues in the Gram matrix in Appendix B.1.

In practice, we can fully mitigate this instability using a restarting strategy. Instead of running all five iterations of the loop in a single pass and outputting $\mathbf{Q}_5 \mathbf{X}$, we run only the first two iterations and compute $\mathbf{X}_2 = \mathbf{Q}_2 \mathbf{X}$. We then “restart” the algorithm treating $\mathbf{X}_2$ as the new input; we construct the Gram matrix $\mathbf{X}_2 \mathbf{X}_2^\top$, initialize $\mathbf{Q}_2 = \mathbf{I}$, and proceed with the remaining three iterations of the loop. This resets any spurious negative eigenvalues to near zero and restores commutativity of $\mathbf{X}$, $\mathbf{Q}$, and $\mathbf{R}$ at the cost of $3(\alpha - 1)n^3$ FLOPs. We find this strategy is sufficient to preserve training quality.

To select the best iteration after which to restart, we sweep for the restart location that provides the best bound on the condition number of $\mathbf{Q}_t$, assuming that forming the Gram matrix introduces spurious eigenvalues as negative as $-4 \cdot 10^{-4}$. We walk through an example of how to automate this sweep in Appendix B.2.

Algorithm 3 presents our stabilized, training-ready version of Gram Newton-Schulz, which uses the restart strategy as well as a slight reformulation of the intermediate polynomials and a switch to `float16` instead of `bfloat16`. The latter two changes are motivated in Appendices B.2.2 and B.2.3. Our algorithm is implemented in a pip-installable package called `gram-newton-schulz`, which uses the Polar Express coefficients [3].

Algorithm 3 Stabilized Gram Newton-Schulz

Input: $\mathbf{X} \in \mathbb{R}^{n \times m}$ with $n \leq m$, coefficients $\{(a_t, b_t, c_t)\}_{t=1}^5$

1: $\mathbf{X} \leftarrow \mathbf{X} / (\ \mathbf{X}\ _F + \epsilon)$	▷ Normalize sing vals to $[0, 1]$, $\epsilon = 10^{-7}$
2: $\mathbf{X} \leftarrow \text{float16}(\mathbf{X})$	▷ Cast to half precision for speed
3: $\mathbf{R}_0 \leftarrow \mathbf{X}\mathbf{X}^\top$	
4: $\mathbf{Q}_0 \leftarrow \mathbf{I}$	
5: for $t = 1, \dots, 5$ do	
6: if $t = 3$ then	▷ Restart to stabilize
7: $\mathbf{X} \leftarrow \mathbf{Q}_2 \mathbf{X}$	
8: $\mathbf{R}_2 \leftarrow \mathbf{X}\mathbf{X}^\top$	
9: $\mathbf{Q}_2 \leftarrow \mathbf{I}$	
10: end if	
11: $\mathbf{Z}_t \leftarrow b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$	
12: $\mathbf{Q}_t \leftarrow \mathbf{Q}_{t-1} \mathbf{Z}_t + a_t \mathbf{Q}_{t-1}$	▷ $\mathbf{Q}_t = \mathbf{Q}_{t-1} h_t(\mathbf{R}_{t-1})$
13: $(\mathbf{RZ})_t \leftarrow \mathbf{R}_{t-1} \mathbf{Z}_t + a_t \mathbf{R}_{t-1}$	
14: $\mathbf{R}_t \leftarrow \mathbf{Z}_t (\mathbf{RZ})_t + a_t (\mathbf{RZ})_t$	▷ $\mathbf{R}_t = \mathbf{R}_{t-1} h_t(\mathbf{R}_{t-1})^2$
15: end for	
16: $\mathbf{X} \leftarrow \mathbf{Q}_5 \mathbf{X}$	
17: return $\mathbf{X}$	

³Transformers’ MLP blocks typically have an intermediate dimension $4 \times$ the model’s hidden dimension.

5 Symmetric GEMM Kernels in CuteDSL

Our second contribution is a set of custom GPU kernels for the operations $\mathbf{AB}$ and $\alpha\mathbf{AB} + \beta\mathbf{C}$ that assume $\mathbf{AB}$ and $\mathbf{C}$ are symmetric. These *symmetric GEMM* kernels compute the lower triangle of the output matrix in the usual way and copy the results to the upper triangle (Figure 2, left), saving about half the floating point operations used by standard matrix multiplication routines. Our kernels allow us to take advantage of the symmetric structure of Gram Newton-Schulz. As noted above, they also accelerate standard Newton-Schulz, but their impact is far greater when combined with Gram Newton-Schulz. We target the Hopper and Blackwell GPU architectures. Figure 2 (right) shows that our kernels achieve superb performance on both architectures, significantly outperforming the standard GEMM routine from cuBLAS across a range of matrix sizes.

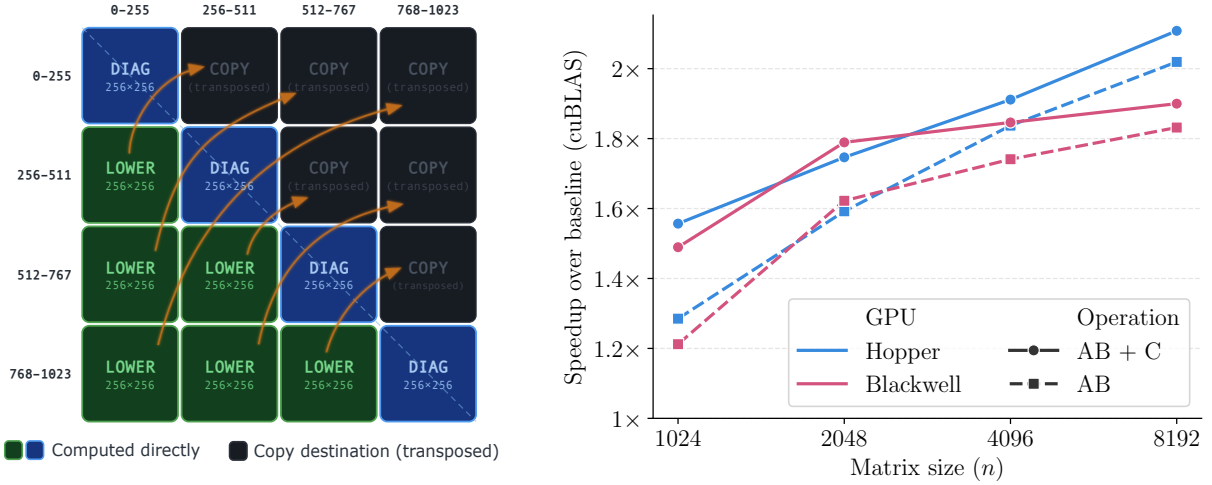


Figure 2: **Left:** Symmetric GEMM computes 256×256 tiles from the lower triangle and main diagonal, then transposes and copies each lower tile to the corresponding upper tile. **Right:** Our CuteDSL symmetric GEMM kernels benchmarked against cuBLAS GEMM kernels on Hopper and Blackwell GPUs. Input matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ have dimensions $n \times n$. For large enough n , our kernels achieve a $\sim 2\times$ speedup over cuBLAS, both with and without an epilogue addition of $\mathbf{C}$.

Most GEMM kernels have the following form:

1. Scheduler: The output matrix is divided into tiles. A schedule is created that assigns each tile to a group of workers.
2. Computing each output tile of $\mathbf{AB}$ or $\alpha\mathbf{AB} + \beta\mathbf{C}$ requires three steps:
 - (a) Prologue: The rows of $\mathbf{A}$ and columns of $\mathbf{B}$ needed for the current tile are loaded in from general memory (high-bandwidth memory) to shared memory (SRAM).
 - (b) Matrix-Multiply Accumulate (MMA): The rows and columns are multiplied and written to the register file (Hopper) or tensor memory (Blackwell).
 - (c) Epilogue: Additional tensors needed for fused operations (e.g. $\mathbf{C}$, α , β) are loaded, fused operations are executed, and the final output is written from the register file to shared memory and then to general memory.

Our symmetric GEMM kernels differ from the standard one only in their schedulers and epilogues.

Triangular Scheduler In the standard GEMM, the entire output matrix is partitioned into work tiles that are load balanced and evenly partitioned amongst clusters of thread blocks, where thread blocks in the same cluster can access the same shared memory and are therefore scheduled to run together. Each cluster then computes its assigned work tiles in succession. Our tile scheduler in the symmetric GEMM is almost identical, except that it only partitions work tiles from the lower triangle of the matrix (including the main

diagonal); work tiles in the upper triangle are unassigned. This *triangular scheduler* ensures that clusters are load balanced and no unnecessary work is performed.

Epilogue: Writing to the Transposed Tile In the GEMM epilogue, when the computed values of the lower triangle (excluding the main diagonal) are written to their assigned tile in general memory (HBM), they are also written to their transposed tile location in the upper triangle. The left panel of Figure 2 illustrates this process.

We describe the details of our symmetric kernels and of our kernelized implementation of standard Newton-Schulz in Appendix C.

6 The Dion3 Update Rule

Our third contribution is a variant of Muon that avoids orthogonalizing the full momentum matrix, making the optimizer even faster. At each step, we select only a fraction of the rows from the momentum matrix and perform the Muon update (including the orthogonalization) as if the other rows did not exist. In words, the main steps of the Dion3 update rule are as follows:

1. **Select** a fraction of the rows (or columns) of the momentum matrix.
2. **Orthogonalize** the selected submatrix using Gram Newton-Schulz.
3. **Update** the selected rows of the weight matrix using the output of the previous step. Do not update the other rows.
4. **Error Feedback** Decay *only* the selected rows of the momentum matrix by a multiplicative factor.

Unlike our other contributions, this procedure changes the optimization trajectory, but experiments in Section 8.1 show that this change is benign. When *all* rows are selected (i.e., the fraction equals 1), our update rule reduces to Muon in exact arithmetic. Note that an earlier version of this update rule appeared in an unpublished report under the name Dion2. As it was not formally published, we drop the old name and present it here as part of Dion3. We now describe each step of the optimizer in detail. Algorithm 4 gives the full pseudocode.

Algorithm 4 Dion3 update rule (single weight matrix)

Input: Weight $\mathbf{W} \in \mathbb{R}^{n \times m}$, gradient $\mathbf{G}$, momentum buffer $\mathbf{M}$, row-wise 2nd moment buffer $\mathbf{v} \in \mathbb{R}^n$, fraction $f \in (0, 1]$, momentum μ , decay β_2 , learning rate η .

- 1: $\mathbf{M} \leftarrow \mathbf{M} + \mathbf{G}$
- 2: $\mathcal{S} \leftarrow$ indices of the $k = \lceil fn \rceil$ rows of $\mathbf{M}$ with largest ℓ_1 norm ▷ Selection
- 3: $\mathbf{O} \leftarrow \text{polar}(\mathbf{M}[\mathcal{S}, :])$ ▷ Orthogonalize $k \times m$ submatrix
- 4: $\mathbf{v}_i \leftarrow \beta_2 \mathbf{v}_i + (1 - \beta_2) \frac{1}{m} \sum_j \mathbf{O}_{ij}^2$ for $i \in \mathcal{S}$ ▷ Optional: NorMuon steps...
- 5: $\hat{\mathbf{O}}_{ij} \leftarrow \mathbf{O}_{ij} / (\sqrt{\mathbf{v}_i} + \epsilon)$
- 6: $\hat{\mathbf{O}} \leftarrow \hat{\mathbf{O}} \|\mathbf{O}\|_F / \|\hat{\mathbf{O}}\|_F$
- 7: $\mathbf{W} \leftarrow (1 - \eta \cdot \text{wd}) \mathbf{W}$ ▷ Optional: weight decay
- 8: $\mathbf{W}_{ij} \leftarrow \mathbf{W}_{ij} - \eta \hat{\mathbf{O}}_{ij}$ for $i \in \mathcal{S}$ ▷ Update selected rows
- 9: $\mathbf{M}_{ij} \leftarrow \mu \mathbf{M}_{ij}$ for $i \in \mathcal{S}$ ▷ Error Feedback

Selection Our update rule introduces a new hyperparameter $f \in (0, 1]$ that controls the fraction of rows (or columns) to select. We think of f as a compression factor; $f = 1$ corresponds to Muon, and decreasing f speeds up the algorithm. We recommend $f = 1/4$ or $f = 1/8$. We could subselect either the rows or the columns. When the weight matrix is sharded across one of these dimensions, we subselect along that same dimension. Otherwise, we pick whichever dimension is smaller. For brevity, we refer only to row selection throughout this section.

We use a simple selection strategy: we pick the $k = \lceil fn \rceil$ rows of largest ℓ_1 norm. Initial experiments showed that optimization quality when using random selection was not much worse, so other selection strategies may perform well. In the distributed setting, selecting the top rows across all shards would require an extra

synchronization round and a ragged all-to-all, so we simply select the top- f fraction of rows from each shard. True global selection is available as an option in our package.

Orthogonalization The benefit of our selection strategy is realized in this step. For an input of size $n \times m$, each matrix multiplication in Newton-Schulz costs either mn^2 or n^3 FLOPs with our symmetric kernels. Selection reduces the dimensions to $fn \times m$, slashing the cost of these multiplications by a factor of at least $1/f^2$. This benefit is compounded by using Gram Newton-Schulz, which is faster than standard Newton-Schulz precisely when the aspect ratio $\alpha = m/n$ is large. Happily, selection increases α by a factor of $1/f$.

Our optimizer also benefits the *communication* cost of Muon in the distributed setting. Rather than assemble the entire momentum matrix on a single device to perform the orthogonalization, we only assemble the selected rows, reducing the communication volume by a factor of $1/f$. We describe our communication strategy in detail in [Section 7](#).

Weight Update We experiment with two flavors of the update rule: Muon and NorMuon (corresponding to (1) and (2), respectively). For the Muon flavor, we use each row of the orthogonalized submatrix $\mathbf{O} = \text{polar}(\mathbf{M}[\mathcal{S}, :])$ to update the matching row of the weight matrix: $\mathbf{W}[\mathcal{S}, :] \leftarrow \mathbf{W}[\mathcal{S}, :] - \eta \mathbf{O}$. The NorMuon flavor is similar, but with extra steps that update the row-wise second moments and use them to rescale $\mathbf{O}$; see [Algorithm 4](#). In both flavors, if weight decay is enabled, it is applied to all rows of $\mathbf{W}$ before updating the selected rows.

Our initial implementation of Dion3 suffered from a nefarious numerical issue that caused it to underperform Muon and NorMuon in final loss. Normally, the compiler fuses the weight update into a single operation $\mathbf{W} \leftarrow (1 - \eta \cdot \text{wd})\mathbf{W} - \eta \mathbf{O}$. Tensors are cast from `bfloat16` to `float32`, the arithmetic is performed, and the result is cast back to `bfloat16`. When we introduced the row selection operator, it broke this automatic fusion, creating multiple rounds of upcasting and downcasting. To restore the original numerical behavior, we implemented a custom Triton kernel for [Lines 7 and 8 of Algorithm 4](#). Furthermore, we found that NorMuon’s normalization step should be performed in `float32` to avoid similar issues.

Error Feedback We adapt the *error feedback* mechanism from Dion [2] to our simpler selection-based strategy for approximating $\mathbf{M}$. The usual rule for updating the momentum matrix is $\mathbf{M} \leftarrow \mathbf{M} + \mathbf{G}$; $\mathbf{M} \leftarrow \mu \mathbf{M}$, which ensures that $\mathbf{M}$ is an exponential moving average of the gradients across iterations. The main idea of error feedback is to decay only the component of $\mathbf{M}$ that was captured by our approximation. Decompose $\mathbf{M}$ into two parts, the selected part $\widehat{\mathbf{M}}$ —which matches $\mathbf{M}$ at the selected rows and contains zeros in all other rows—and the unselected part $\mathbf{M} - \widehat{\mathbf{M}}$. (In Dion, $\widehat{\mathbf{M}}$ would be a low-rank approximation of $\mathbf{M}$.) Instead of the usual rule $\mathbf{M} \leftarrow \mu \mathbf{M}$, error feedback does $\mathbf{M} \leftarrow \mu \widehat{\mathbf{M}} + (\mathbf{M} - \widehat{\mathbf{M}})$. That is, it decays the selected rows by a factor of μ but leaves the other rows alone.

In effect, error feedback adds an extra $(1 - \mu)(\mathbf{M} - \widehat{\mathbf{M}})$ to the momentum. By boosting the residual component, this extra term nudges future iterations to select rows that were ignored by the current iteration. Even if a given row of $\mathbf{G}$ is consistently small, error feedback allows it to build up in $\mathbf{M}$ over many iterations, so it eventually gets selected and participates in the update. When the approximation is exact ($\mathbf{M} = \widehat{\mathbf{M}}$), we recover standard momentum.

Finite Precision At $f = 1$ every row is selected each step, and Dion3 reduces to Muon/NorMuon up to four implementation details: the selection step permutes the rows, momentum damping is applied after forming the update⁴, the NorMuon normalization steps run in `float32`, and the update step uses our custom Triton kernel. We confirmed that, despite these differences, the convergence curve of our optimizer matches that of Muon / NorMuon almost exactly (see [Appendix D.4](#)).

Kernel launch minimization Dion3’s row selection process necessarily requires more kernel launches than standard Muon does. At small scale, the cost of these extra kernel launches can dominate. Since the

⁴That is, Newton-Schulz sees $\mathbf{M} + \mathbf{G}$ rather than $\mu \mathbf{M} + \mathbf{G}$. Since $\mathbf{M}$ is initialized to zero, these matrices differ by a constant factor of μ , which is removed by orthogonalization anyway.

kernel dependency structure is the same from round to round, we use a standard CUDA graph capture and replay to avoid this overhead and realize the full potential of our optimizer on smaller-scale models.

7 Megabatching and Communication

Our final contribution concerns the distributed setting. Under fully-sharded data parallelism (FSDP), each momentum matrix is partitioned across GPUs, so it must first be assembled onto a single device before Newton-Schulz can run. This requires an all-to-all communication along the sharded dimension and a second all-to-all to scatter back the result. As discussed in [Section 2.3](#), the cost of these operations is a major obstacle to scaling Muon [\[2, 11\]](#). To help users easily adopt Dion3 in a wide range of scenarios, we implement it in `dion`, a pip-installable package that gracefully manages this communication for FSDP2, DDP, and mixed sharding strategies with minimal overhead. The package supports our kernelized implementation of Gram Newton-Schulz and both flavors of the Dion3 update rule as well as Muon, NorMuon, and standard Newton-Schulz, all of which share the same distributed backend. We believe `dion` provides a full-stack, production-ready solution for distributed orthogonal optimization.

A key feature of our implementation is **megabatching**. While the update rule of [Section 6](#) reduces the communication *volume* (the number of bits that must be assembled and scattered), megabatching reduces the number of *rounds* of communication. A naive implementation of Muon performs the update in batches of `world_size` matrices at a time; each matrix in the batch is assembled and orthogonalized in parallel on a different GPU without any GPU sitting idle. For a model with N orthogonalized matrices, this requires $O(N/\text{world_size})$ rounds of communication (all-to-alls) per optimizer step. Unfortunately, launching and synchronizing each round incurs overhead. Since this overhead is independent of the communication volume, our update rule does not mitigate it. Furthermore, each peer-to-peer message is small—just one shard of one parameter. These messages may fail to saturate the interconnect, so bandwidth utilization is poor. We demonstrate these effects in [Appendix D.3](#).

Our solution is megabatching; we group all matrices of the same shape into a *single* batch. For a given shape, the local momentum shards are packed into one all-to-all, assembled, orthogonalized as a batch, and scattered back together. Transformers contain only a handful of distinct weight shapes, so megabatching reduces the number of communication rounds to $O(1)$, independent of model depth. We implement this megabatching strategy in the `dion` package, where it powers Muon, NorMuon, and Dion3 alike. Note that megabatching only changes *when* data moves; the assembly, momentum-state layout, checkpoint format, and orthogonalization math are all the same.

Benchmarking We compare megabatching against the previous communication strategy by measuring the wall-clock time of the optimizer step. We hold the model, data, and optimizer (Muon) fixed, and time both strategies back-to-back on the same GPUs. We test two model sizes (1B and 14B parameters) and two FSDP configurations (8 or 32 GPUs). Results are shown in [Table 1](#). For the 14B model, the computational cost of Newton-Schulz dominates, so reducing communication overhead has little impact. For the 1B model with 32 shards, each rank holds only one or two matrices of each shape anyway (see [Table 4](#)), so the batch size is about the same for both strategies. However, for the 1B model with 8 shards, the optimizer is communication-bound and each rank holds many matrices. Here, megabatching has a major impact, reducing step time by 35%.

Table 1: Effect of megabatched communication on the per-GPU optimizer step time (Muon, median over steps). “Batching” orthogonalizes matrices in groups whose size equals the shard count; “megabatch” uses a single batch per shape group. When the weights are small (so Newton-Schulz is cheap) and each rank holds several matrices, megabatching substantially reduces optimizer time. Each node has four GH200s.

Model size	Nodes	FSDP Shards	Batching (ms)	Megabatching (ms)	Change
1B	1	8	80.7	52.1	−35%
1B	4	32	61.9	59.3	−4%
14B	1	8	144.0	140.8	−2%
14B	4	32	94.7	89.1	−6%

Compressed Data Parallelism There is a prospect for further savings at larger scales, where data parallelism spans multiple pods in a data-center network (DCN) or hybrid sharding is used [35]. A common deployment combines model parallelism within an inter-chip interconnect (ICI) with pure data parallelism across pods [4]. Because DCN bandwidth is typically far smaller than ICI bandwidth, shrinking the volume of data-parallel communication is especially valuable in this regime. Here, Dion3 allows for compressed data-parallel synchronizations analogous to those of DION [2, §3.3]. Ordinarily, gradients are averaged across data-parallel replicas at every step so that the momentum buffers, and hence the weights, stay consistent. With Dion3, however, we need not synchronize the entire momentum matrix at every step, only the selected submatrix $\mathbf{M}[\mathcal{S}, :]$. When the rows $\mathcal{S}$ can be selected without first synchronizing $\mathbf{M}$ (e.g., by picking $\mathcal{S}$ at random), this strategy performs an identical update at a fraction of the communication cost.

8 Experiments

8.1 Model Quality Is Preserved

Of our four contributions, only our update rule explicitly changes the optimizer trajectory. However, Gram-Newton-Schulz and our symmetric GEMM kernels introduce numerical differences due to finite precision arithmetic. Experiments in Appendix A.2 show that these differences do not affect training. We also confirmed that, as expected, Dion3 with selection fraction $f = 1$ matches Muon/NorMuon almost exactly (see Figure 22).

In contrast, Dion3 with $f < 1$ is a genuinely new optimizer. In this section, we demonstrate that Dion3 achieves a slightly *better* loss than the baseline. We did not set out to improve training quality, so this result is a small but pleasant surprise. At a minimum, it shows that our subselection strategy does no harm to the loss.

We train 1B-parameter dense transformers on 100B tokens of the ClimbMix dataset [10], using fully-sharded data parallelism (FSDP) with MXFP8 weights. (Below, we also train at larger scales.) As always, Newton-Schulz runs in half precision arithmetic. The models use grouped-query and sliding-window attention, but are *not* mixtures of experts. We report the cross-entropy loss (CE) of next-token prediction on a held-out validation split of ClimbMix. For further details about our setup, see Appendix D.1. For a fair comparison, we begin by tuning the baseline. In this section, we compare NorMuon to the NorMuon flavor of Dion3, as initial experiments gave it a slight edge over Muon. We find that the optimal learning rate and momentum coefficient for NorMuon are $\eta = 0.01$ and $\mu = 0.9$, respectively.

Learning-rate transfer rule Since Dion3 updates only a fraction of the rows in each step, it reduces the effective step size. This in turn changes the optimal learning rate. To understand this effect, we sweep both the fraction $f \in \{1/2, 1/4, 1/8, 1/16\}$ and the learning rate η of Dion3, along with our earlier sweep for NorMuon ($f = 1$). The results in Figure 3 show a clear trend: the optimal learning rate for a given f scales as $\eta \propto \sqrt{1/f}$. We can explain this transfer rule as follows. For Muon, the size of the update is $\|\eta \cdot \text{polar}(\mathbf{M})\|_F = \eta\sqrt{n}$, where n is the smaller dimension. For Dion3, it is $\|\eta' \cdot \text{polar}(\mathbf{M}[\mathcal{S}, :])\|_F = \eta'\sqrt{fn}$. For these to match, we must set $\eta' = \eta/\sqrt{f}$. We do not bother retuning the momentum coefficient; we simply copy the optimal setting from NorMuon ($\mu = 0.9$) to Dion3.

Dion3 improves the loss Figure 3 contains another remarkable finding: Dion3 with $f < 1$ actually outperforms NorMuon when tuned correctly. Indeed, the lowest loss is achieved at $f = 1/8$. Figure 4 shows the validation loss curve for each optimizer (excluding $f = 1/4$) at its best learning rate. It shows a clear gap throughout training between NorMuon and the Dion3 variants, which confirms this finding.

To strengthen this finding, we scale up the model size to between 3B and 14B parameters. For reasons of cost, we now train on 10B tokens instead of 100B. We compare Dion3 with $f = 1/4$ against NorMuon, each using their best learning rate as identified above. For each scale and optimizer, Table 2 reports both validation loss on ClimbMix and downstream accuracy on a suite of 12 standard benchmarks. In validation loss, Dion3 outperforms NorMuon at every scale, achieving the largest improvement (-0.027) at the largest scale (14B). Figure 5 shows that, as for the 1B parameter model, validation loss is lower throughout training when using Dion3. In downstream accuracy, Dion3 wins at three of four scales, with an improvement of 0.7 percentage points at 14B.

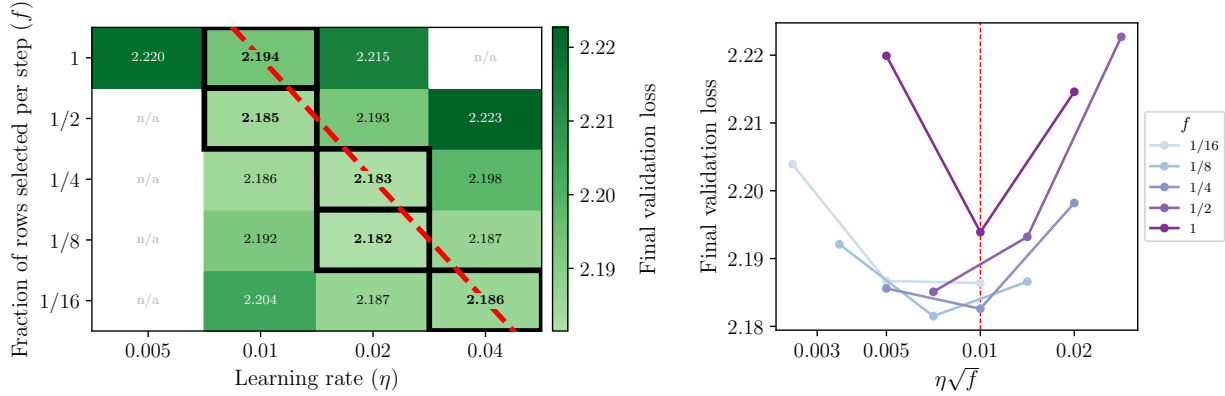


Figure 3: **Left:** Final validation loss for 1B-parameter models trained on 100B tokens of ClimbiMix with NorMuon or Dion3, as a function of row-selection fraction f and learning rate η . NorMuon is $f = 1$. Bolded cell in each row shows best η . Optimal η s track the line $\eta\sqrt{f} = 0.01$, shown in red on a log-log scale. Not shown: $f = 1, \eta = 0.03$ (loss = 2.237) and $f = 1/32, \eta = 0.04$ (loss = 2.191). **Right:** The same data reduced to two dimensions: validation loss as a function of $\eta\sqrt{f}$. Final loss is minimized when $\eta\sqrt{f} \approx 0.01$.

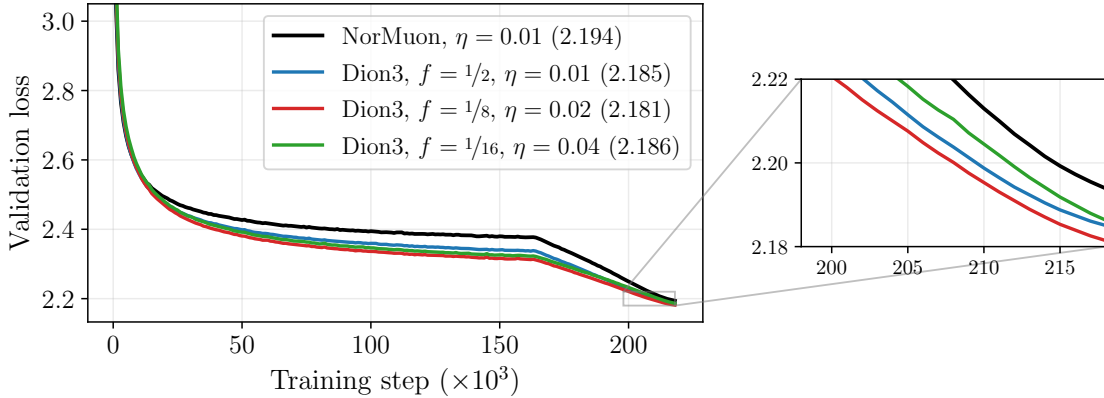


Figure 4: Validation loss curves for 1B-parameter models trained on 100B tokens of ClimbiMix with NorMuon or Dion3 at $f = 1/2, 1/8$, or $1/16$. Each optimizer uses its best learning rate from Figure 3. Parentheses show final loss. All Dion3 variants track below the fully-tuned NorMuon baseline throughout training, finishing about 0.01 points lower.

8.2 Dion3 Accelerates the Optimizer

We now measure how much Dion3 reduces the cost of Muon’s optimizer step. Starting with standard Muon, we successively add our improvements—symmetric kernels, Gram Newton-Schulz, and fractional updates with $f = 0.5$ or $f = 0.25$ —stacking each on top of the previous ones. (All versions use megabatching.) We also compare against plain AdamW. We run each version of the optimizer on models of various sizes either on 1 GPU or using FSDP on 4 GPUs. We use CUDA events to measure the GPU time of the optimizer step. For fidelity, we run full training steps with forward and backward passes (on synthetic data) but these are excluded from the recorded timings.

Results are shown in Figure 6. A subset of these results appears in Figure 1, and the corresponding timings for the NorMuon family are given in Appendix D.4 (Figure 23). Each part of Dion3 yields a consistent speedup across scales and parallelism configurations. Our symmetric kernels and Gram Newton-Schulz give a combined speedup of $1.5\times$ or more over standard Muon. The fractional update rule cuts the runtime dramatically. As expected, its impact grows with model size, as the cubic computational cost of Newton-Schulz increasingly dominates the other operations. For sufficiently large models, setting $f = 1/2$ and $f = 1/4$ gives further $2\times$ and $3.7\times$ reductions respectively, for overall speedups of $3.6\times$ and $6.5\times$. Thus, while Dion3 is still slower

Table 2: Dion3 ($f = 1/4$, $\eta = 0.02$) versus NorMuon ($\eta = 0.01$) across model sizes trained on 10B tokens of ClimbMix: final validation loss (cross-entropy; lower is better) and downstream accuracy (macro-average over 12 standard benchmarks: ARC easy/challenge, BoolQ, COPA, HellaSwag, LAMBADA, MMLU, OpenBookQA, PIQA, RTE, TruthfulQA, WinoGrande; higher is better). The winner of each comparison is bolded.

Model Size	Validation Loss			Downstream Accuracy (%)		
	NorMuon	Dion3 ($f = 1/4$)	Δ	NorMuon	Dion3 ($f = 1/4$)	Δ
3B	2.269	2.257	-0.012	53.9	54.9	+1.0
4B	2.243	2.232	-0.011	55.2	54.9	-0.3
7B	2.220	2.206	-0.014	56.0	56.1	+0.1
14B	2.189	2.162	-0.027	57.4	58.1	+0.7

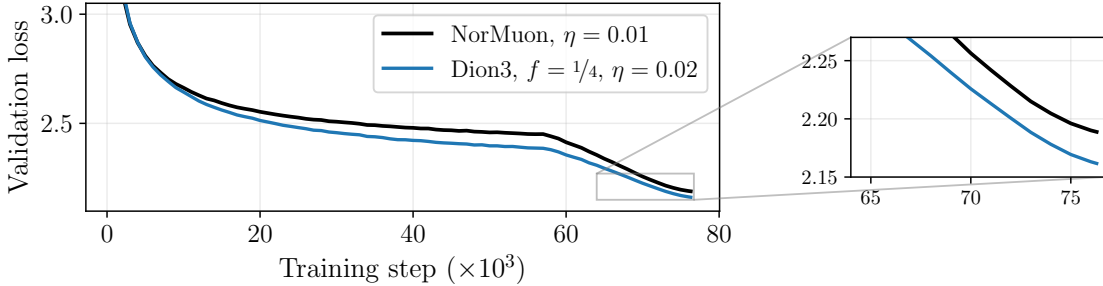


Figure 5: Validation loss over training at 14B for Dion3 ($f = 1/4$, $\eta = 0.02$) versus NorMuon ($\eta = 0.01$), trained on 10B tokens of ClimbMix. Dion3 tracks below NorMuon throughout.

than AdamW, the gap is *significantly* smaller.

In [Appendix D.2](#), we report CPU time and communication volume in addition to GPU time, showing that CPU overhead is small and that fractional updates reduce communication by a factor of $1/f$. The communication savings of our fractional update rule can be crucial in some settings, though the setting of [Figure 6](#) is compute-bound: neither communication nor host time is exposed.

In [Appendix A](#), we conduct an additional suite of benchmarks on a wider range of architectures, including open source models like Gemma and mixtures-of-experts. On architectures like these, whose weight matrices have higher aspect ratios ($\alpha = 8$ instead of 4), Gram Newton-Schulz and the symmetric kernels alone achieve a speedup of $2\times$ (see [Figures 9](#) and [10](#)).

9 Conclusion

For LLMs trained with Muon, estimates suggest that optimizer step time accounts for between 1% (e.g., Abadji et al. [\[1\]](#)) and 17% of total training time (see [Appendix E](#)). While Muon has been scaled successfully, doing so has demanded particular choices of architecture and parallelism strategy, along with *significant* engineering effort. As interest in Muon continues to spread, this paradigm is too brittle; a flexible, general-purpose remedy for Muon’s overhead is needed.

Dion3 allows practitioners to easily realize the benefits of Muon without paying the high cost of its orthogonalization step—even for large-scale models in highly distributed settings. Our Gram Newton-Schulz algorithm and CuteDSL kernels speed up Muon by $1.5\times$ for dense models and $2\times$ for MoEs, a rare case of free lunch performance. Megabatching has an equally large effect in certain distributed settings. Dion3’s fractional update rule ($f = 1/4$) provides an *additional* $3.7\times$ speedup. Though this update rule changes the optimizer trajectory, we find that it actually improves training quality in our setting. This improvement is unexpected, as we designed Dion3 to cheaply approximate Muon rather than improve upon it. Further work is needed to determine how widely this improvement generalizes, but we find support in a recent result similar to our own: Joo et al. [\[18\]](#) showed that randomly masking blocks of the update improves the trajectory of SGD with momentum. Overall, Dion3 as implemented in our `dion` and `gram-newton-schulz` packages provides

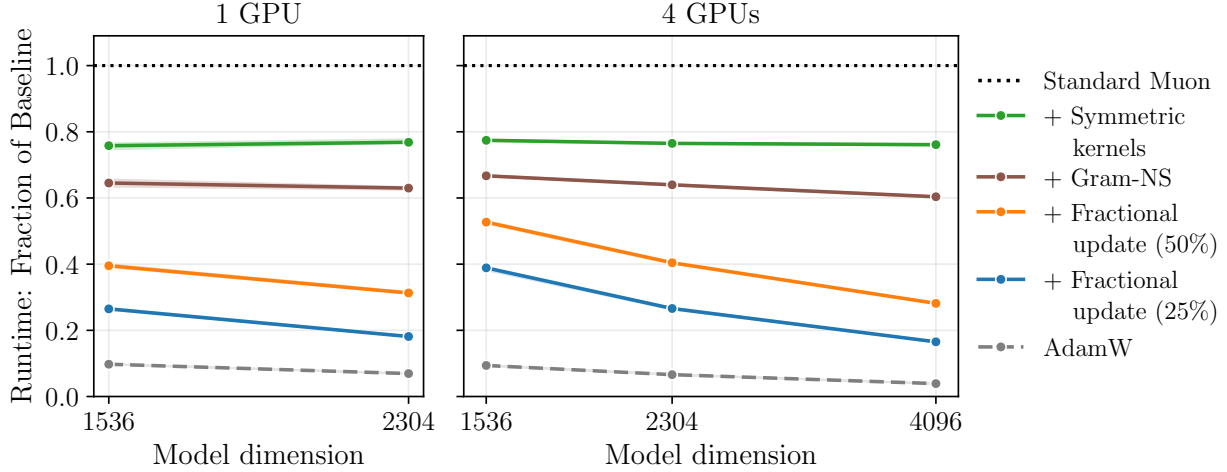


Figure 6: Optimizer step time (excluding forward/backward pass) relative to standard Muon across model scales when training on 1 GH200 (left) and FSDP over 4 GH200s (right). Each colored line adds one of our contributions on top of the previous one; AdamW is shown for reference. Lines show median over 25 steps and bands show interquartile range. Adding each of our contributions consistently reduces runtime; together they achieve a $6\times$ speedup for larger models.

the tools to make orthogonal optimizers practical and accessible in a wide range of settings.

Acknowledgements The authors thank Zichong Li for contributing an implementation of NorMuon to the dion repo. NA is supported by NSF award 2234660.

References

- [1] Julien Abadji, Marah Abdin, Connor Adams, Eric Alcaide, Mustafa Altun, Michele Artoni, Junze Bao, Uday Barar, Vassilis Bekiaris, Arkadii Bessonov, et al. *Laguna M.1/XS.2 Technical Report*. 2026. arXiv: [2605.27605](#).
- [2] Kwangjun Ahn, Byron Xu, Natalie Abreu, Ying Fan, Gagik Magakyan, Pratyusha Sharma, Zheng Zhan, and John Langford. *Dion: Distributed Orthonormalized Updates*. 2025. arXiv: [2504.05295](#).
- [3] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. “The Polar Express: Optimal Matrix Sign Methods and their Application to the Muon Algorithm”. In: *The Fourteenth International Conference on Learning Representations*. 2026. <https://openreview.net/forum?id=yRtgZ1K8h0>.
- [4] Jacob Austin, Sholto Douglas, Roy Frostig, Anselm Levskaya, Charlie Chen, Sharad Vikram, Federico Lebron, Peter Choy, Vinay Ramasesh, Albert Webson, and Reiner Pope. *How to Scale Your Model*. Google DeepMind, 2025. <https://jax-ml.github.io/scaling-book/>.
- [5] Jeremy Bernstein. *Deriving Muon*. 2025. <https://jeremybernste.in/writing/deriving-muon>.
- [6] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. *Turbo-Muon: Accelerating Orthogonality-Based Optimization with Pre-Conditioning*. 2025. arXiv: [2512.04632](#).
- [7] Valentyn Boreiko, Zhiqi Bu, and Sheng Zha. “Towards understanding of orthogonalization in Muon”. In: *Tiny Titans: The next wave of On-Device Learning for Foundation Models (TTODLer-FM)*. 2025. <https://openreview.net/forum?id=4vzhqq5hpX>.
- [8] Franz Louis Cesista, Jiacheng You, and Keller Jordan. *Squeezing 1-2% Efficiency Gains Out of Muon by Optimizing the Newton-Schulz Coefficients*. 2025. <http://leloykun.github.io/ponder/muon-opt-coeffs/>.

- [9] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. *DeepSeek-V3 Technical Report*. 2025. arXiv: [2412.19437](https://arxiv.org/abs/2412.19437).
- [10] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Celine Lin, Jan Kautz, and Pavlo Molchanov. “Nemotron-CLIMB: Clustering-based Iterative Data Mixture Bootstrapping for Language Model Pre-training”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2025. <https://openreview.net/forum?id=aBlqKPk4a>.
- [11] Essential AI. *Layer Sharding for Large-Scale Training with Muon*. 2025. <https://www.essential.ai/research/infra>.
- [12] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivi  re, et al. *Gemma 3 Technical Report*. 2025. arXiv: [2503.19786](https://arxiv.org/abs/2503.19786).
- [13] GLM-5 Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, et al. *GLM-5: from Vibe Coding to Agentic Engineering*. 2026. arXiv: [2602.15763](https://arxiv.org/abs/2602.15763).
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. *The Llama 3 Herd of Models*. 2024. arXiv: [2407.21783](https://arxiv.org/abs/2407.21783).
- [15] Ekaterina Grishina, Matvey Smirnov, and Maxim Rakhuba. *Accelerating Newton-Schulz Iteration for Orthogonalization via Chebyshev-type Polynomials*. 2026. arXiv: [2506.10935](https://arxiv.org/abs/2506.10935).
- [16] Wentao Guo, Mayank Mishra, Xinle Cheng, Ion Stoica, and Tri Dao. *SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations*. 2026. arXiv: [2512.14080](https://arxiv.org/abs/2512.14080).
- [17] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, 2008. DOI: [10.1137/1.9780898717778](https://doi.org/10.1137/1.9780898717778).
- [18] Taejong Joo, Wenhan Xia, Cheolmin Kim, Ming Zhang, and Eugene Ie. *On Surprising Effectiveness of Masking Updates in Adaptive Optimizers*. 2026. arXiv: [2602.15322](https://arxiv.org/abs/2602.15322).
- [19] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. *Muon: An optimizer for hidden layers in neural networks*. 2024. <https://kellerjordan.github.io/posts/muon/>.
- [20] Ahmed Khaled, Kaan Ozkara, Tao Yu, Mingyi Hong, and Youngsuk Park. “MuonBP: Faster Muon via Block-Periodic Orthogonalization”. In: *The Fourteenth International Conference on Learning Representations*. 2026. <https://openreview.net/forum?id=mHouLSUQP5>.
- [21] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. *Kimi K2: Open Agentic Intelligence*. 2025. arXiv: [2507.20534](https://arxiv.org/abs/2507.20534).
- [22] Slobodan Laki  . “On the Computation of the Matrix k-th Root”. In: *Zeitschrift f  r Angewandte Mathematik und Mechanik* 78.3 (1998), pp. 167–172. <https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291521-4001%28199803%2978%3A3%3C167%3A%3AAID-ZAMM167%3E3.0.CO%3B2-R>.
- [23] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. “NorMuon: Making Muon more efficient and scalable”. In: *Forty-third International Conference on Machine Learning*. 2026. <https://openreview.net/forum?id=m1lRWFAMsa>.
- [24] Junghwan Lim, Sungmin Lee, Dongseok Kim, Taehyun Kim, Eunhwan Park, Jeessoo Lee, Jeongdoo Lee, Junhyeok Lee, Wai Ting Cheung, Dahye Choi, et al. *Motif 2 12.7B technical report*. 2025. arXiv: [2511.07464](https://arxiv.org/abs/2511.07464).
- [25] Tianyang Lin. *Flash-Muon: An Efficient Implementation of Muon Optimizer*. 2025. <https://github.com/nil0x9/flash-muon>.
- [26] Jingyuan Liu. *A proof of concept for Distributed Muon*. Pull Request #1428, NVIDIA/Megatron-LM. Accessed: 2026-07-10. 2025. <https://github.com/NVIDIA/Megatron-LM/pull/1428>.
- [27] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. *Muon is Scalable for LLM Training*. 2025. arXiv: [2502.16982](https://arxiv.org/abs/2502.16982).
- [28] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. *FineWeb-Edu: the Finest Collection of Educational Content*. 2024. DOI: [10.57967/hf/2497](https://doi.org/10.57967/hf/2497).

- [29] Ionut-Vlad Modoranu, Mher Safaryan, Erik Schultheis, Max Ryabinin, Artem Chumachenko, and Dan Alistarh. “Trion: FFT-based Dynamic Subspace Selection for Low-Rank Adaptive Optimization of LLMs”. In: *The Fourteenth International Conference on Learning Representations*. 2026. <https://openreview.net/forum?id=TkHjRwbMN1>.
- [30] Laker Newhouse, Dakota Goldberg, and Ricardo Ruiz. *Faster symmetric matrix multiplication with ThunderKittens*. 2024. <https://www.lakernewhouse.com/assets/writing/faster-symm-mul-with-thunderkittens.pdf>.
- [31] OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, et al. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: [2508.10925](https://arxiv.org/abs/2508.10925).
- [32] Ziyuan Tang, Tianshi Xu, Yousef Saad, and Yanzhe Xi. *Hierarchical Muon: Tiled Newton-Schulz Updates for Efficient Muon Optimization*. 2026. arXiv: [2606.27216](https://arxiv.org/abs/2606.27216).
- [33] Byron Xu. *Transpose one of the MLP matrices + add Triton kernel for symmetric matmul*. Pull Request #109, KellerJordan/modded-nanogpt. Accessed: 2026-07-10. 2025. <https://github.com/KellerJordan/modded-nanogpt/pull/109>.
- [34] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. *Qwen3 Technical Report*. 2025. arXiv: [2505.09388](https://arxiv.org/abs/2505.09388).
- [35] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel”. In: *Proceedings of the VLDB Endowment* 16.12 (2023), pp. 3848–3860.

Appendices

A Alternative Experiments on Gram Newton-Schulz

This section presents additional experimental results about Gram Newton-Schulz and the symmetric GEMM kernels. Complementing [Section 8](#), these experiments study a wider range of architectures but at smaller scale, using only one GPU at a time.

A.1 Setup

We train four architectures: Llama-430M, Qwen-600M, Gemma-1B, and a custom MoE architecture with 1B parameters, of which $\sim 20\%$ are active [\[12, 14, 34\]](#). We train on FineWeb-Edu [\[28\]](#). The number of training tokens for each dense model is given by the Chinchilla scaling law; for MoE-1B it is twice the Chinchilla scaling law with respect to the active parameters. We use a cosine learning rate scheduler with the base learning rates given in [Table 3](#).

We use Muon on most matrix parameters—the attention layer’s projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$), the projection following attention ($\mathbf{W}_O$), the SwiGLU MLP weights ($\mathbf{W}_{\text{MLP,UP}}, \mathbf{W}_{\text{MLP,GATE}}, \mathbf{W}_{\text{MLP,DOWN}}$), and the token router of the MoE ($\mathbf{W}_{\text{router}}$)—but exclude the embedding and unembedding layers. Although we are not in the distributed setting, we use megabatching ([Section 7](#)), as batching Newton-Schulz’s GEMM operations makes them faster. As is standard for Muon, we adjust the learning rate by scaling the update for each weight matrix based on its dimensions. For these experiments, we found that Moonshot AI’s strategy of scaling by $0.2\sqrt{\max(\text{fan_out}, \text{fan_in})}$ yields the best loss curves [\[27, §2.2\]](#).

Table 3: Architecture, base learning rate, and training-token budget of each model.

Model	Hidden dim	Layers	MLP dim	Learning rate	Training tokens
Llama-430M	1024	24	4096	3×10^{-3}	10B
Qwen-600M	1024	28	3072	1.5×10^{-3}	12B
Gemma-1B	2048	8	16384	3×10^{-4}	22B
MoE-1B	1024	9	(per-expert) 256	2.5×10^{-3}	11B

A.1.1 Splitting the Weights

We draw special attention to the fact that we split $\mathbf{W}_{\text{MLP,UP}}$ from $\mathbf{W}_{\text{MLP,GATE}}$ and orthogonalize them separately. Ordinary implementations of SwiGLU MLPs concatenate these into a single weight matrix; however, their contributions to the activation are fundamentally different, so gradients are different too. We find that orthogonalizing them separately improves the final loss; for example, in Llama-430M, we observe an improvement of ≈ 0.2 in perplexity. For MoE architectures—whose intermediate size is typically smaller than the hidden size—separating them also reduces the FLOP cost of orthogonalization by a factor of two for standard Newton-Schulz and even more for Gram Newton-Schulz. Likewise, while earlier implementations of Muon orthogonalized the combined matrix $[\mathbf{W}_Q \mid \mathbf{W}_K \mid \mathbf{W}_V]$, we orthogonalize each piece separately.

We are also aware that in some settings, including pretraining of GLM-5, Muon benefits from splitting the Multi-Latent Attention weights ($\mathbf{W}^{UQ}, \mathbf{W}^{UK}$, and $\mathbf{W}^{UV}$) by attention head before orthogonalizing [\[13\]](#). This choice is principled, since the actual function computed by attention treats each head separately; it does not see the concatenated weights. Inspired by this, we experimented with splitting $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, and $\mathbf{W}_O$ by attention head to form H matrices each of size $\frac{d}{H} \times d$, where d is the embedding dimension and H is the number of heads. However, this design led to higher losses throughout training, so we did not adopt it.

Still, we believe that there are other settings like GLM-5 where this strategy works well. Such cases would benefit *immensely* from Gram Newton-Schulz, since the aspect ratio of these weight matrices would be the number of heads H . For a standard attention weight like $\mathbf{W}_Q$ with $H = 16$ and $T = 5$, Gram Newton-Schulz on the little matrices would use **80×** fewer FLOPs than orthogonalizing the big matrix!

A.2 Kernelized Gram Newton-Schulz preserves quality

We first verify that switching from standard Newton-Schulz to our kernelized implementation of Gram Newton-Schulz does not affect training quality. We try each implementation with both the Polar Express coefficients [3] and those of Cesista et al. [8]. As Figure 7 shows, the loss curves are identical in all cases, and the final validation perplexity is preserved to within 0.01. Figure 7 uses a Hopper GPU, but we got the same results on Blackwell.

We see loss preserved as follows, when both using the Polar Express coefficients and the coefficients derived by Cesista et al. [8]:

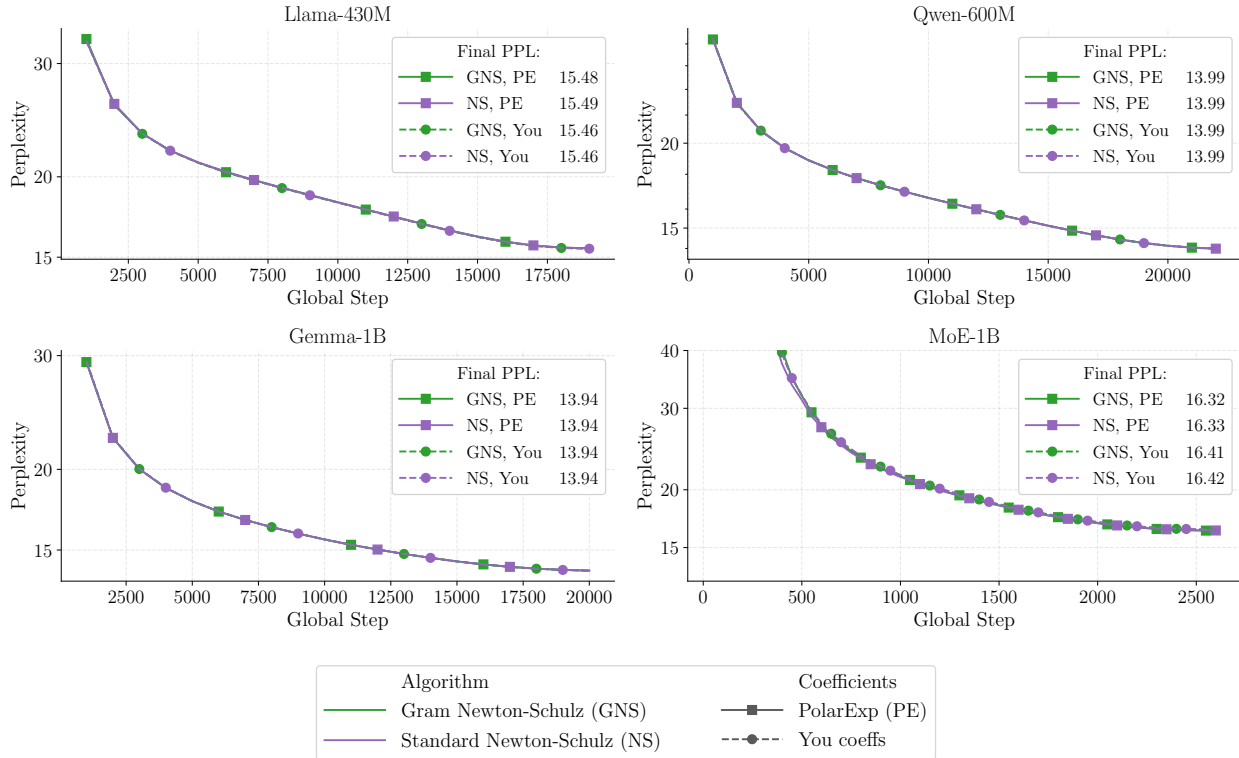


Figure 7: When training with Muon on a Hopper GPU, switching from standard Newton-Schulz to Gram Newton-Schulz preserves the validation perplexity throughout training (up to a 0.01 difference in GNS’s favor). Setup is described in Appendix A.1.

A.3 Kernelized Gram Newton-Schulz speeds up the optimizer step

Newton-Schulz Performance We observe that our method speeds up the runtime of the Newton-Schulz step by 1.5–2 $\times$. Figure 8 reports these speed-ups for each model, benchmarked on both H100 and B300 GPUs. As expected, savings are greatest for weights that are highly rectangular, like Gemma’s MLP weights ($\alpha = 8$) and MoE-1B’s expert weights ($\alpha = 4$). Note that these experiments use standard Newton-Schulz as the fallback when $m = n$.

End-to-End Optimizer Performance Figure 9 shows the end-to-end wall clock time of the Muon optimizer step for each version of Newton-Schulz, along with that of AdamW. For Muon, these timings include updating the momentum matrix, learning rate scaling, applying the weight update, and the AdamW updates for weights not assigned to Muon (such as the embedding layer and the vector-valued weights). Our method yields a 1.3–2 $\times$ speedup, with both Gram Newton-Schulz and the kernels contributing significantly.

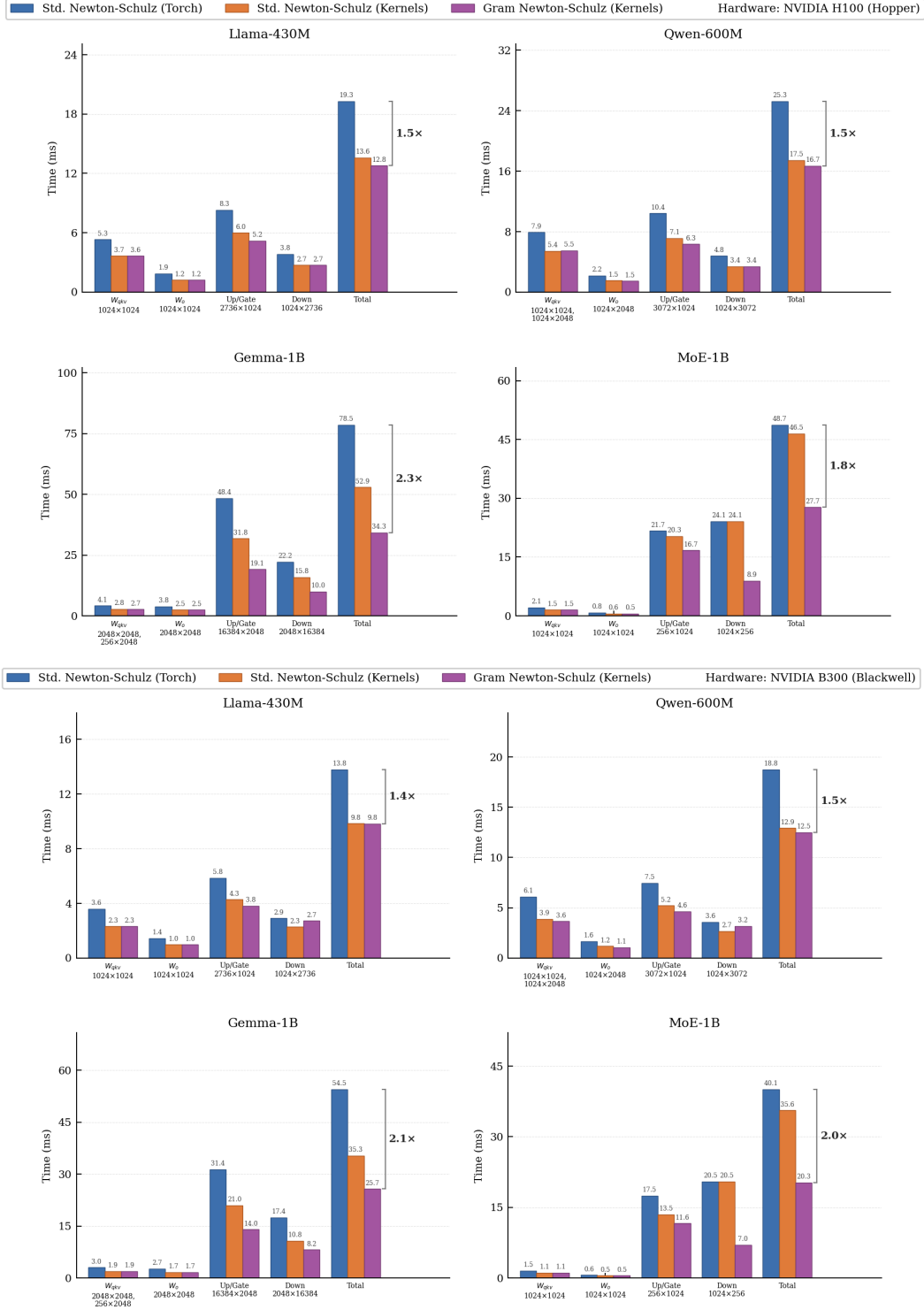


Figure 8: Newton-Schulz time per model weight for (1) standard Newton-Schulz implemented in pure PyTorch, (2) standard Newton-Schulz with our symmetric GEMM kernels, and (3) Gram Newton-Schulz with our kernels. We test four architectures on a single Hopper (top) or Blackwell (bottom) GPU. Square weights (W_Q or W_K) benefit from the kernels. Rectangular weights—especially those with a high aspect ratio (e.g. Up/Gate in Gemma-1B)—additionally benefit from Gram Newton-Schulz.

As before, the impact of Gram Newton-Schulz is greatest for architectures with highly rectangular weights. Due to its MLP’s higher $8\times$ aspect ratio, Gemma-1B sees the largest speedup.

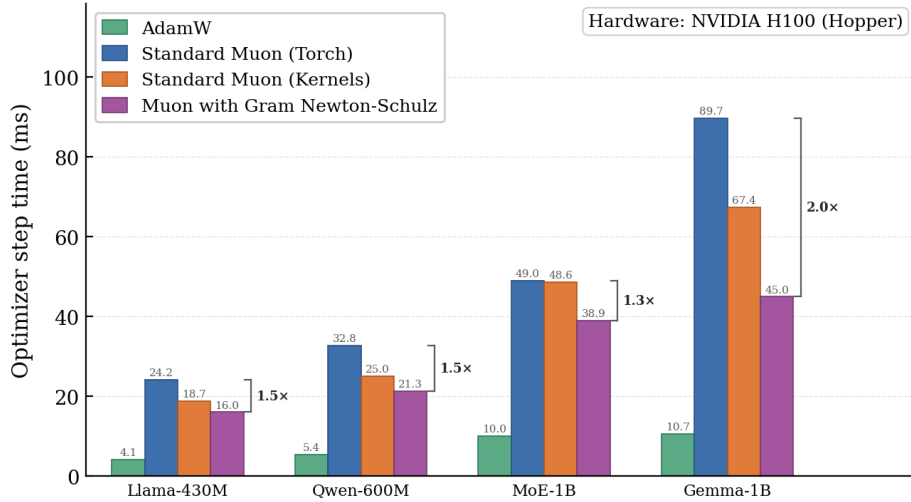


Figure 9: Optimizer step time for AdamW and Muon with three different Newton-Schulz routines on a single H100. Gram Newton-Schulz with our symmetric kernels gives a 1.3–2 $\times$ speedup over standard Newton-Schulz. Timings include matrix splitting and recombination for QKV and MLP, learning rate scaling, weight updates, and the scalar optimizer (AdamW) step for non-2D weights.

Estimating Gram Newton-Schulz time in Kimi K2 Kimi K2 is a trillion parameter sparse, fine-grained MoE model with 384 experts per layer, a hidden size of 7168, and a small expert intermediate dimension of 2048. Since models are trending towards finer-grained MoE architectures and Kimi K2 was trained with Muon, this is a perfect setting to benchmark Gram Newton-Schulz. Due to Kimi’s sophisticated pipeline parallel strategy, the optimizer steps of many of its weights are completely hidden behind the backward pass of the next pipeline stage. We estimate that orthogonalization steps of following weights are exposed: 216 expert up/gate/down weights of shape 2048×7168 and 1 dense up/gate/down weight of shape 7168×18432 . Therefore, we estimate the exposed Newton-Schulz time by orthogonalizing weights of these shapes on a single GPU. Figure 10 shows the results: our method yields a 2 $\times$ speedup. As before, both our kernels and the Gram Newton-Schulz algorithm contribute significantly.

B Stability of Gram Newton-Schulz

B.1 Instability of Naive Gram Newton-Schulz

We trained Llama-430M with Muon using Naive Gram Newton-Schulz (Algorithm 2). The loss curve is shown in Figure 11. Clearly, training is very unstable. Not only do we observe loss spikes, but eventually, the outputs of Gram Newton-Schulz become full of Infs. The problem is due to floating point arithmetic. While Gram Newton-Schulz is mathematically equivalent to standard Newton-Schulz in exact arithmetic, it behaves differently in finite precision, especially in half precision. As half precision is essential for good performance, this makes Naive Gram Newton-Schulz unworkable in practice.

We now analyze the source of the instability in order to motivate our solution. A Jupyter notebook reproducing the experiments in this section is available [here](#).

B.1.1 Tracking Eigenvalues of Intermediate Matrices

To understand how the matrices in Naive Gram Newton-Schulz evolve and why they diverge, we track their eigenvalues and singular values. Recall that the entries of any matrix are upper bounded by its largest

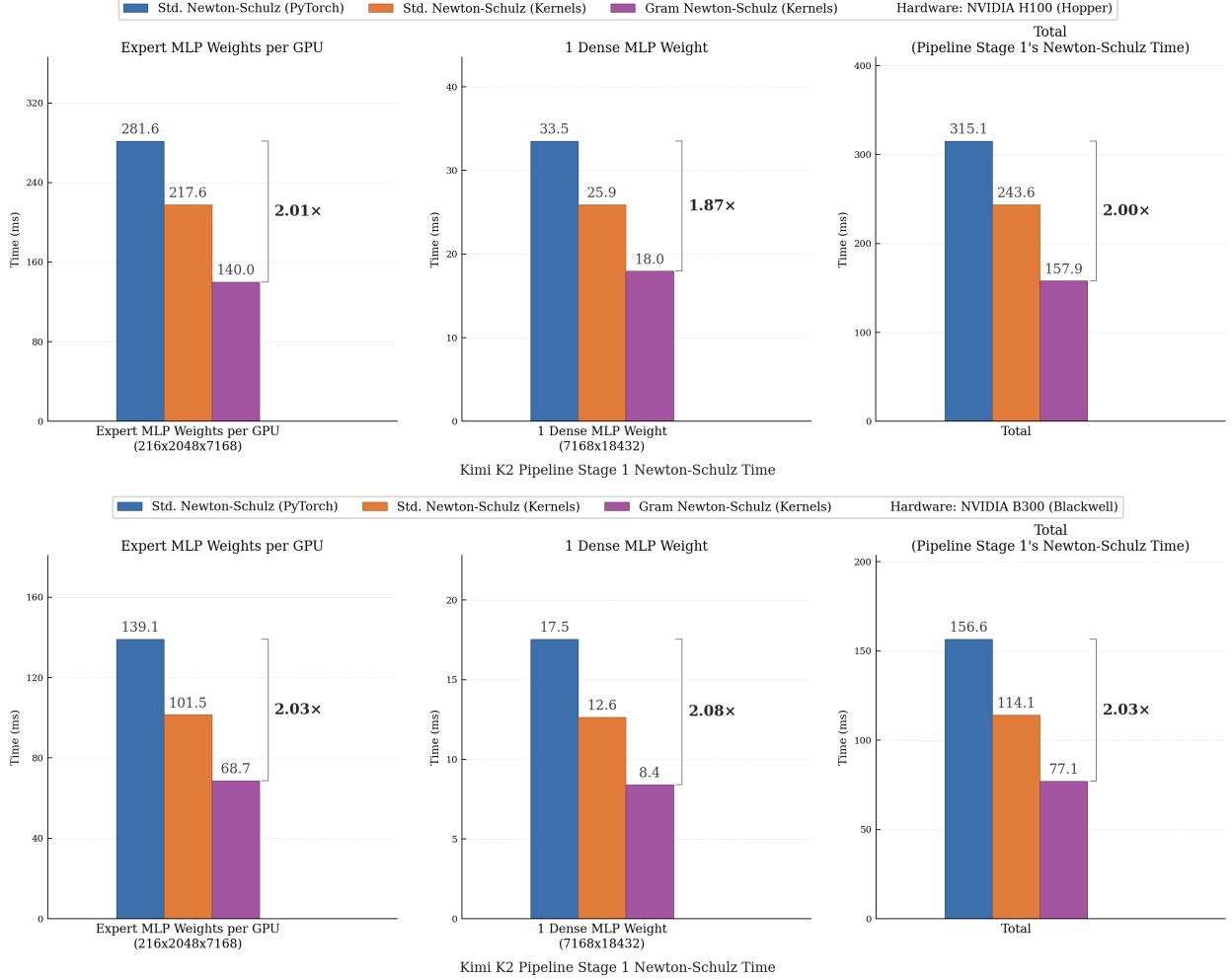


Figure 10: Estimated exposed Newton-Schulz time for one step of Kimi K2 with pipeline parallelism. We measure the runtime of the exposed operations on a single Hopper (top) or Blackwell (bottom) GPU. Gram Newton-Schulz with kernels is $2\times$ faster than standard Newton-Schulz.

singular value, so if we can bound the singular values, we can prevent blowups.

As a baseline, we start by running Naive Gram Newton-Schulz in full `float64` precision for 8 steps to simulate its behavior in exact arithmetic. We use a synthetic 128×512 matrix with an exponentially decaying spectrum. To make our plots more readable, the experiments in this section use the coefficients $(a_t, b_t, c_t) = (\frac{15}{8}, -\frac{10}{8}, \frac{3}{8})$ at every iteration, but our conclusions will generalize to the coefficients used in practice. If $\mathbf{X}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the SVD of the input matrix, then the intermediate matrices of Algorithm 2 ($\mathbf{R}_t, \mathbf{Q}_t, \mathbf{Z}_t$) are square symmetric with eigenvectors $\mathbf{U}$. We therefore plot the diagonal entries of $\mathbf{U}^\top \mathbf{R}_t \mathbf{U}$ and $\mathbf{U}^\top \mathbf{Q}_t \mathbf{U}$ against the corresponding singular values in $\mathbf{\Sigma}$ to track how each evolves according to the polynomial update rules—or diverges from them. Even though Gram Newton-Schulz does not need to compute $\mathbf{X}_1, \dots, \mathbf{X}_{T-1}$, we do so here for demonstration using the formula $\mathbf{X}_t = \mathbf{Q}_t \mathbf{X}_0$ and plot $\mathbf{U}^\top \mathbf{X}_t \mathbf{V}$ against $\mathbf{\Sigma}$. The top panel of Figure 12 shows that the eigenvalues evolve as predicted by Theorem 2. Initially, $x_0 \in [0, 1]$, $r_0 = x_0^2 \in [0, 1]$ and $q_0 = 1$. As the algorithm progresses, $r_t \rightarrow 1$, $q_t = x_t/x_0 \rightarrow 1/x_0$, and $x_t \rightarrow 1$. For $x_0 \approx 1$, convergence is faster; for $x_0 \ll 1$, it is slower.

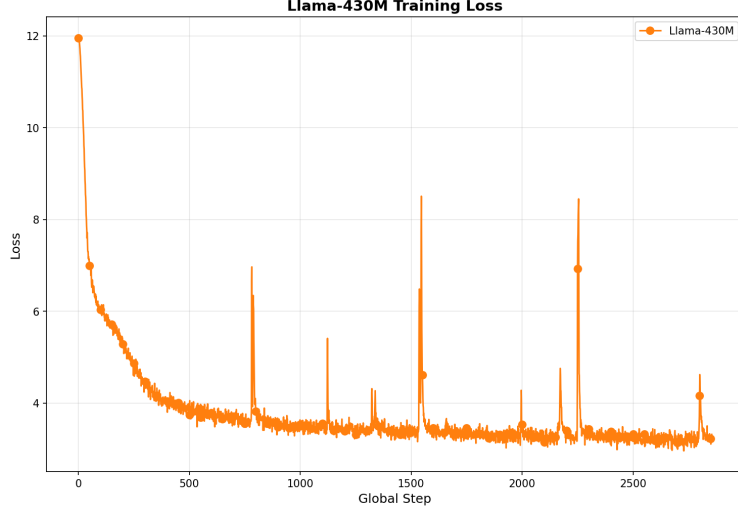


Figure 11: Naive Gram Newton-Schulz used to train Llama-430M in half precision. Numerical instability wrecks training.

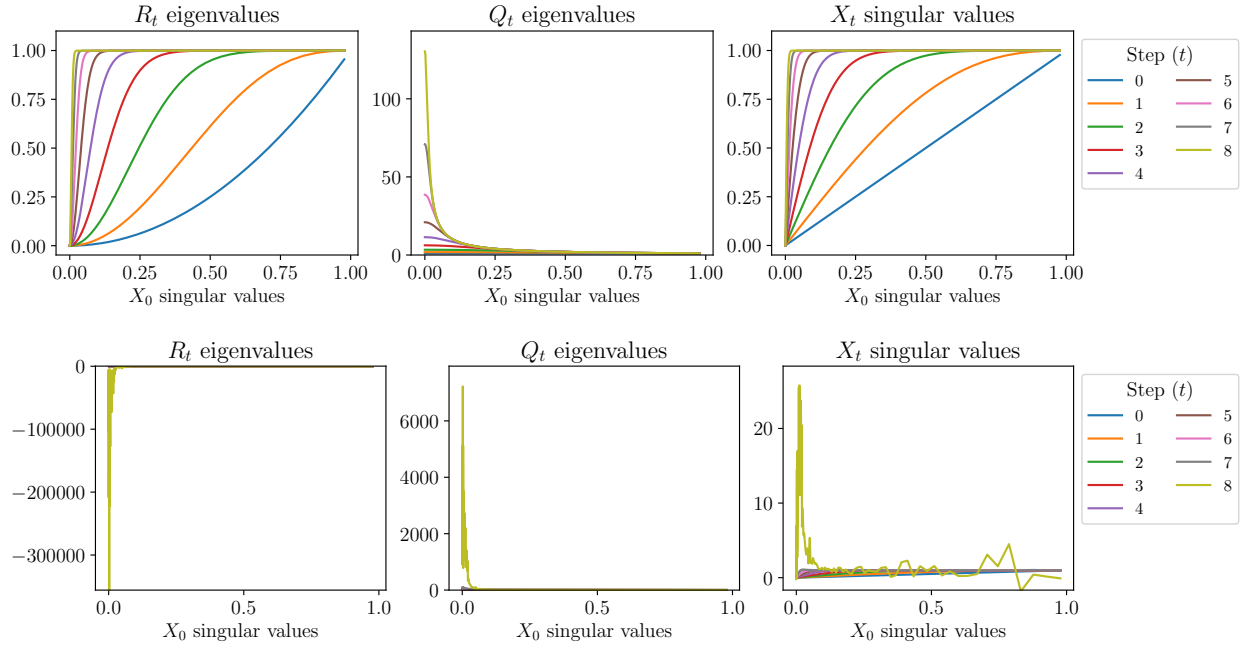


Figure 12: Evolution of eigenvalues of $\mathbf{R}_t$, $\mathbf{Q}_t$, and $\mathbf{X}_t$ in Naive Gram Newton-Schulz with $(a_t, b_t, c_t) = (15/8, 10/8, 3/8)$ as a function of the corresponding singular value of $\mathbf{X}_0$. **Top:** float64. **Bottom:** bfloat16.

Now we rerun the experiment in bfloat16 arithmetic (Figure 12, bottom). The first few iterations look correct, but by step 7, we see noisy and unexpected behavior ($r_t < 0$, $x_t > 1$). By step 8, the spectrum diverges completely, and by step 10 the algorithm returns Infs. We now describe two key causes of divergence: spurious negative eigenvalues in the Gram matrix $\mathbf{X}\mathbf{X}^\top$, and eigenvector drift.

B.1.2 Spurious Negative Eigenvalues

The leading cause of divergence is the presence of negative eigenvalues in the Gram matrix due to half-precision arithmetic. These negative eigenvalues blow up after too many iterations of Gram Newton-Schulz.

By construction, $r_t = x_t^2 \geq 0$, so $\mathbf{R}_t$ should be positive semidefinite (Theorem 2), but Figure 12 shows that it has negative eigenvalues. In fact, even $\mathbf{R}_0$ has tiny negative eigenvalues introduced in the first matrix multiplication $\mathbf{X}_0 \mathbf{X}_0^\top$. Because $\mathbf{X}_0$ has many singular values that are nearly zero (as is the case in Muon), $\mathbf{R}_0 = \mathbf{X}_0 \mathbf{X}_0^\top$ has many eigenvalues that are *numerically equal* to zero. In `bf16`, even a slightly negative number can be numerically equal to zero. Later computations can introduce additional negative eigenvalues into $\mathbf{R}_t$ as well. These eigenvalues represent nothing about the original problem; they are just an artifact of floating point arithmetic. Therefore, we call them “spurious eigenvalues”.

These spurious negative eigenvalues start small, but Figure 12 shows that their magnitude grows quickly. Recall the update rule:

$$r_t = r_{t-1} z_t^2 = r_{t-1} h_t(r_{t-1})^2 \quad (4)$$

For our choice of coefficients, $h_t(x) = \frac{15}{8} - \frac{10}{8}x + \frac{3}{8}x^2$. Figure 13 plots this update rule. As it shows, $r_t < (\frac{15}{8})^2 r_{t-1}$. Thus, if any $r_t < 0$, the spurious negative eigenvalues grow exponentially, diverging to $-\infty$! This sets off a chain reaction that causes $\mathbf{Q}_t$ and $\mathbf{X}_t$ to diverge as well. This problem cannot be fixed by choosing different polynomials; while the main loop of Gram Newton-Schulz approximates the inverse square root of $r_0 > 0$, it diverges for negative inputs.

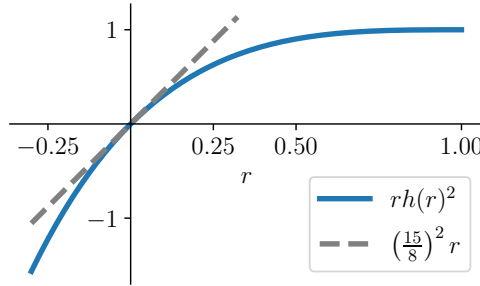


Figure 13: When $\mathbf{R}_t$ evolves according to (4), negative eigenvalues diverge to $-\infty$.

To prove that the tiny spurious negative eigenvalues of $\mathbf{R}_0$ suffice to cause divergence, we rerun the experiment in `float64` precision, but cast $\mathbf{R}_0$ from `float64` to `bf16` and then back to `float64` to induce a small floating point error. As Figure 14 shows, this is enough to cause a blowup.

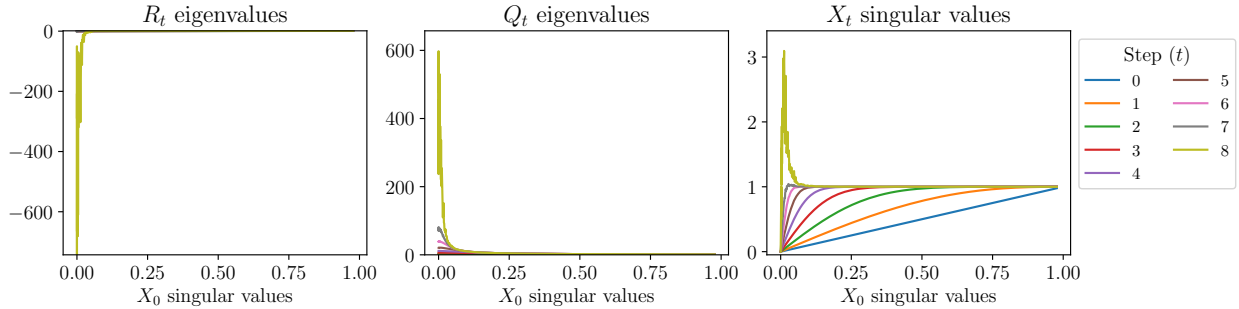


Figure 14: Evolution of eigenvalues of $\mathbf{R}_t$, $\mathbf{Q}_t$, and $\mathbf{X}_t$ when all operations use `float64` except $\mathbf{R}_0 = \mathbf{X}_0 \mathbf{X}_0^\top$, which uses `bf16`.

B.1.3 Eigenvector Drift

Spurious negative eigenvalues are not the only source of numerical instability. If we take the input matrix $\mathbf{X}_0$ to have no small singular values (i.e., all ≥ 0.017), then we do not observe any negative eigenvalues in $\mathbf{R}_t$, but $\mathbf{X}_t$ still fails to converge. The culprit seems to be “eigenvector drift”.

In exact arithmetic, the eigenvectors of all intermediate matrices match $\mathbf{U}$, the left singular vectors of $\mathbf{X}_0$, but in finite precision they do not. We demonstrate this effect by measuring how far $\mathbf{U}^\top \mathbf{R}_t \mathbf{U}$, $\mathbf{U}^\top \mathbf{Q}_t \mathbf{U}$, and $\mathbf{U}^\top \mathbf{X}_t \mathbf{U}$ are from being diagonal matrices. That is, we take the Frobenius norm of the off-diagonal entries as a fraction of the total Frobenius norm. Figure 15 shows that after several iterations, the eigenvectors / singular vectors of $\mathbf{Q}_t$ and $\mathbf{X}_t$ have drifted significantly from those of $\mathbf{X}_0$. At the same time, the eigenvalues of $\mathbf{Q}_t$ (and by extension, those of $\mathbf{X}_t$) diverge from where they would be in exact arithmetic. The growing eigenvalues of $\mathbf{Q}_t$ seem to spill into one another. The strength of this effect is less consistent than that of negative eigenvalues, but it is still harmful.

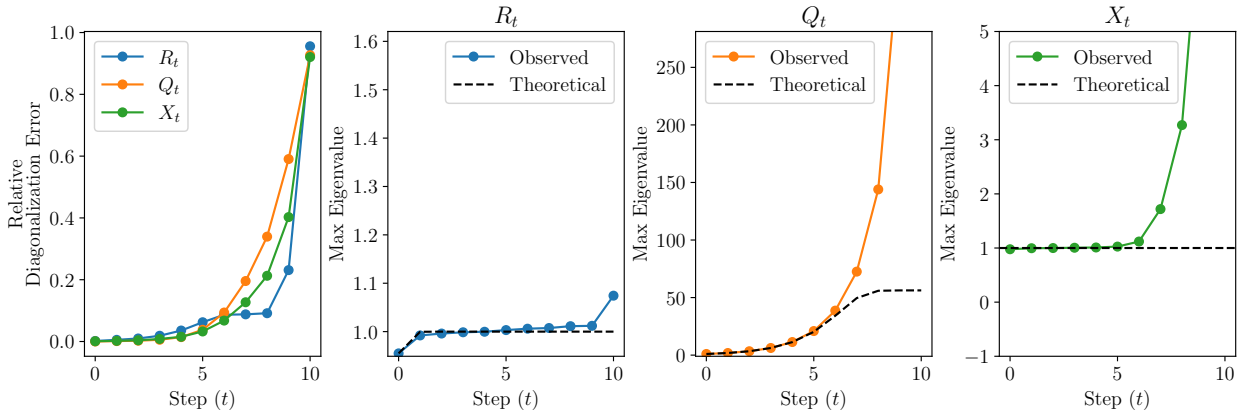


Figure 15: As the eigenvectors drift, the spectral norms of $\mathbf{R}_t$, $\mathbf{Q}_t$, and $\mathbf{X}_t$ diverge.

B.2 Stabilizing Gram Newton-Schulz by Restarting

As we have seen, if we run Gram Newton-Schulz for more than a few iterations, the spurious negative eigenvalues of $\mathbf{R}_t$ diverge to negative infinity and $\mathbf{Q}_t$ blows up. Our solution is simple: run Gram Newton-Schulz for only a few iterations. For instance, rather than using Gram Newton-Schulz to compute $\mathbf{X}_T$ directly, we can use it to compute $\mathbf{X}_5$ in a stable manner. While $\mathbf{X}_5$ is not a good approximation to $\text{polar}(\mathbf{X}_0)$, it is closer than where we started. Now we apply Gram Newton-Schulz a second time on the input $\mathbf{X}_5$ to compute $\mathbf{X}_{10}$ stably. This process can be repeated to reach any desired $\mathbf{X}_T$. This restarting technique sacrifices some of the performance gains of Gram Newton-Schulz, but it still offers a significant speedup over standard Newton-Schulz.

Figure 16 repeats the experiment from above with a restart every five iterations. While $\mathbf{R}_t$ develops some negative eigenvalues, unlike before, the growth of these eigenvalues is controlled. Each time we restart, we re-initialize $\mathbf{R}_t = \mathbf{X}_t \mathbf{X}_t^\top$, eliminating any large negative eigenvalues. By restarting regularly, we prevent any from growing too large. As expected, $\mathbf{Q}_t$ resets to the identity at iterations 5, 10, 15, 20, and 25. Therefore, the eigenvalues of $\mathbf{Q}_t$ never grow beyond ≈ 12 , despite the negative eigenvalues in $\mathbf{R}_t$. Since the eigenvalues of $\mathbf{Q}_t$ remain controlled, those of $\mathbf{X}_t = \mathbf{Q}_t \mathbf{X}_{t-5}$ stay at or below 1.

Restarting also helps control eigenvector drift. We repeat the experiment from Figure 15 on the same matrix (with all singular values > 0.017) with a restart after step 5. Diagonalization error always remains ≤ 0.05 , and the maximum eigenvalues now closely track their predicted values. Note that we always measure eigenvector drift relative to the original input $\mathbf{X}_0$, not the restarted $\mathbf{X}_5$.

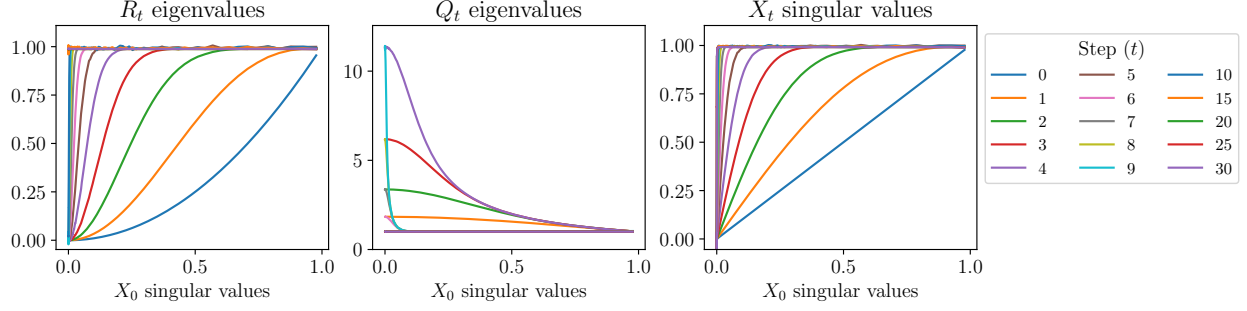


Figure 16: Restarting prevents the divergence of R_t in half-precision.

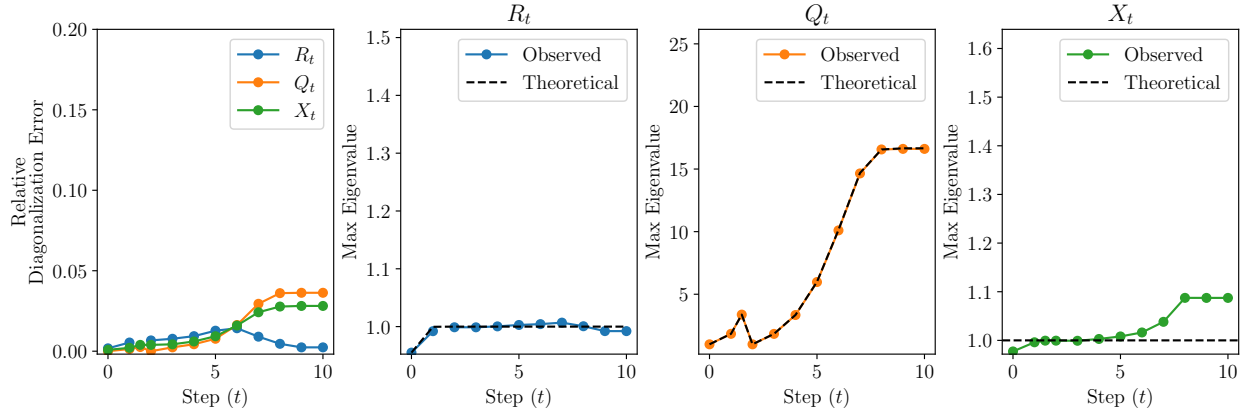


Figure 17: Restarting curbs eigenvector drift.

B.2.1 When to Restart: Polar Express Coefficients for Muon

We now derive the optimal restarting schedule for a given total number of iterations T . To prevent $X_t = Q_t X_0$ from blowing up, we must control the condition number of Q_t , even when R_0 has spurious negative eigenvalues. (Because $q_t \rightarrow 1/x_0 \geq 1$, this also controls the maximum eigenvalue of Q_t .) The growth of Q_t in turn depends on the size of the spurious negative eigenvalues and the specific sequence of polynomials defined by $\{(a_t, b_t, c_t)\}_{t=1}^T$. Fixing a desired number of restarts, we sweep over all possible restart schedules and pick the one that minimizes the maximum condition number of Q_t across ts .

For the application to Muon, we analyze five iterations of the Polar Express coefficients [3]. In the experiments of the previous section, we observe that the most negative spurious eigenvalue of R_0 is about $-4 \cdot 10^{-4}$. Therefore, we simulate Gram Newton-Schulz in full precision with Polar Express coefficients and track how the eigenvalues of R_t and Q_t evolve when R_0 has eigenvalues in the range $[-4 \cdot 10^{-4}, 1]$. The left panels of Figure 18 show that without restarting, they blow up. We then repeat the simulation with one restart, sweeping all possible choices of when to restart. Every time we restart and form $R = XX^\top$, we subtract $4 \cdot 10^{-4}I$ to simulate potentially dangerous corruption of the eigenvalues due to floating point error. As the right panel shows, restarting after the *second* iteration provides the best bounds, ensuring that the eigenvalues of R_t stay well above -0.4 and those of Q_t stay below ≈ 100 for all iterations. Figure 19 shows that for Gram Newton-Schulz with Polar Express coefficients and one restart after the second iteration, all eigenvalues of X_t converge stably to 1.

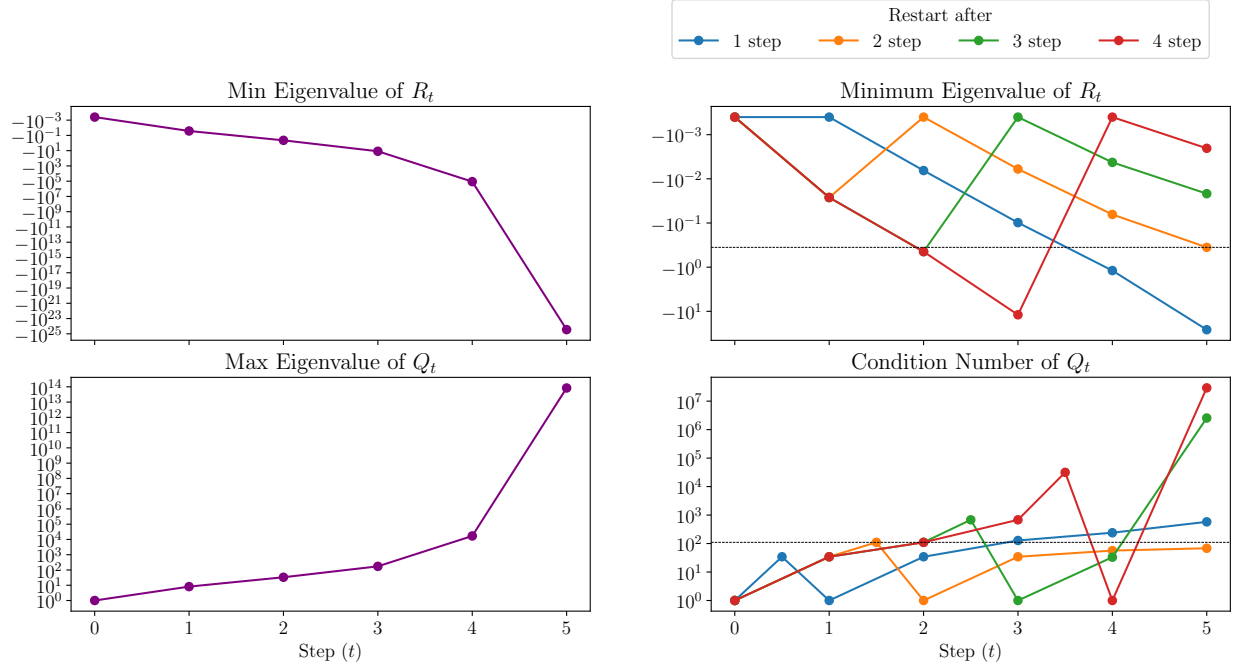


Figure 18: **Left:** Min/max eigenvalue of R_t and Q_t without restarts. R_0 starts with a negative eigenvalue at $-4 \cdot 10^{-4}$. **Right:** Minimum eigenvalue of R_t and condition number of Q_t if restart is placed after iteration 1, 2, 3, or 4. Restarting after iteration 2 best controls the size of Q_t .

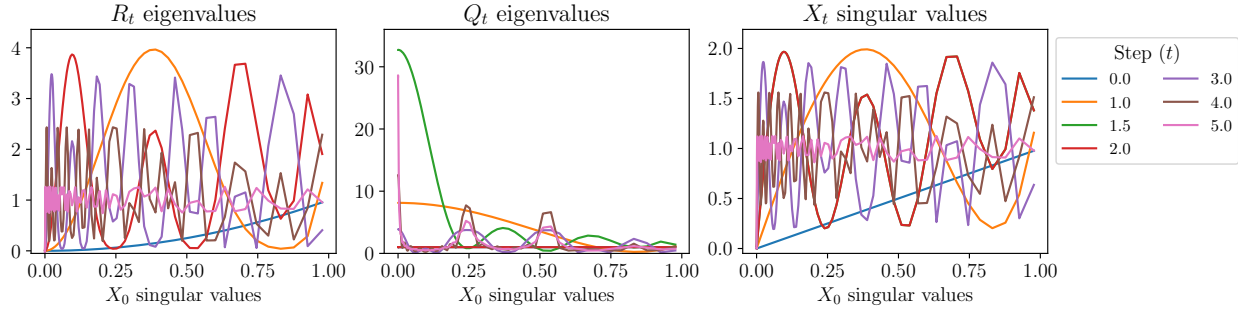


Figure 19: Gram Newton-Schulz with Polar Express coefficients and a restart after 2 iterations converges stably.

B.2.2 Further Precautions

While restarting greatly improves stability, it is not absolutely foolproof. A second or third restart may be required if running for more than five iterations or using a numerically sensitive set of coefficients. Moreover, the usual numerical precautions for standard Newton-Schulz still apply.

Safety Factors Most choices of Newton-Schulz polynomials are designed to converge only when the input lies in $[0, 1]$; any singular values larger than 1 may diverge rapidly, as Figure 20 shows. Even when X_0 is properly normalized, singular values greater than 1 can arise due to numerical error. This problem affects standard Newton-Schulz too, so the Polar Express polynomials are typically adjusted according to the formula $\tilde{p}_t(x) = p_t(x/1.02)$. This ensures convergence even for singular values as large as 1.02. When using Gram Newton-Schulz, roundoff errors like this can worsen due to computations like $\mathbf{X}\mathbf{X}^\top$, which do not have such

numerical buffers; however, we have never seen this happen when using our recommended setup (`float16` arithmetic with restarting after 2 iterations). It is wise to be conservative in the choice of safety factor, for instance, by replacing 1.02 with 1.05.

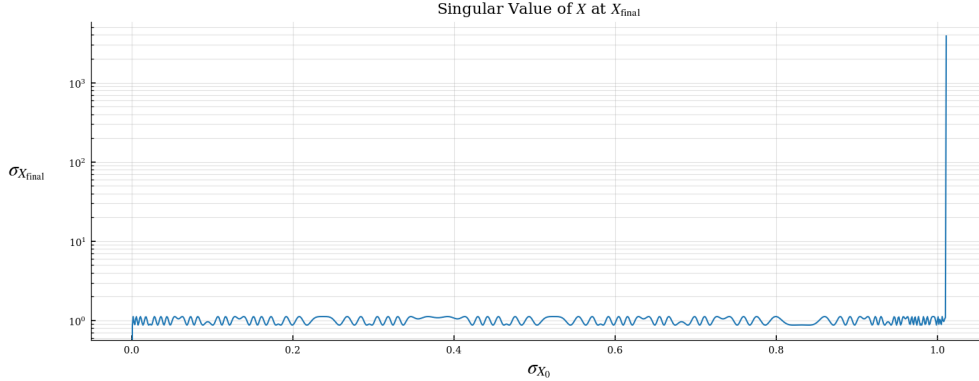


Figure 20: Theoretical behavior of both standard and Gram Newton-Schulz (Polar Express coefficients) when $\mathbf{X}_0$ has singular values slightly above one.

Float16 vs BFloat16 in Newton-Schulz In addition, we argue for using `float16` instead of `bfloat16` to implement Newton-Schulz. Both use 16 bits and thus run with the same performance; however, the distribution of the 16 bits across the mantissa and exponent bits differs. Compared to `bfloat16`, `float16` represents values from a narrower range, but it has greater precision within that range. For Newton-Schulz, however, the range of `float16` (roughly $6.1 \cdot 10^{-5}$ to $6.5 \cdot 10^4$) suffices because the magnitudes of its intermediate matrices are not too large. On certain test matrices, we see more accurate polar($\mathbf{X}$) approximations with `float16`, but in practice, we have not found a case where the training quality is meaningfully different between `float16` and `bfloat16`. Still, we default to `float16`.

High Accuracy Setting In the application to Muon, we do not need to compute polar($\mathbf{X}$) to high accuracy, and [Appendix A.2](#) shows that Muon with Gram Newton-Schulz yields effectively identical results to Muon with standard Newton-Schulz in terms of training quality. However, when high accuracy is desired, the usual warnings about forming the Gram matrix apply. Since forming $\mathbf{X}\mathbf{X}^\top$ immediately squares the condition number, Gram Newton-Schulz may not be appropriate in these cases.

B.2.3 Computing Matrix Quadratics

A key step in Gram Newton-Schulz is computing the matrix quadratic $\mathbf{Z}_t \leftarrow a_t \mathbf{I} + b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$. Standard Newton-Schulz implicitly computes matrix quadratics too, but PyTorch implementations typically avoid adding the identity explicitly. Rather, they compute $\mathbf{X}(a_t \mathbf{I} + b_t \mathbf{A} + c_t \mathbf{A}^2)$ in two steps, each with a single GEMM:

1. $\mathbf{B} \leftarrow b_t \mathbf{A} + c_t \mathbf{A}^2$
2. $\mathbf{X} \leftarrow a_t \mathbf{X} + \mathbf{B}\mathbf{X}$

Our symmetric GEMM kernel can compute matrix quadratics in one launch, fusing the addition of $a_t \mathbf{I}$ by adding a_t to all diagonal entries of the output when they are at the register level. This optimization completely obviates any I/O operations needed for the $a_t \mathbf{I}$ addition, typically outspeeding `gemm_symmetric(A, B, C, c_t, b_t) + a_t * I`, which would require loading $\mathbf{I}$ from general memory to shared memory to registers. Once $\mathbf{Z}_t$ is assembled, Gram Newton-Schulz can perform its three subsequent multiplications without the need for further fused additions.

However, our tests show that adding $a_t \mathbf{I}$ explicitly can be less stable than distributing it into those later three multiplications. If we stress-test our method by ignoring some of our own advice—restarting after three iterations instead of two, using a Polar Express safety factor of 1.02 instead of 1.05, and computing

the quadratic with $a_t \mathbf{I}$ explicitly—we observe instability. Interestingly, this instability disappears if we use non-symmetric GEMMs (either from PyTorch or Quack) instead of our symmetric kernels; however, if we force symmetry after calling standard PyTorch GEMMs, we see instability again. We conclude that fusing $+a_t \mathbf{I}$ into a symmetric GEMM is numerically unfavorable; this is not just a kernel bug.

We believe this effect can be explained as follows. While the fused kernel computes $a_t \mathbf{I} + b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$ in `float32` arithmetic under the hood, the result $\mathbf{Z}_t$ is rounded back down to `float16` at the end of the GEMM. Future computations like $\mathbf{Q}_t \mathbf{Z}_t$ suffer from this loss of precision in a_t . In contrast, if the $a_t \mathbf{I}$ term is handled implicitly, all arithmetic involving a_t takes place in `float32`. Therefore, it is more stable to compute $a_t \mathbf{Q}_t + \mathbf{Q}_t (b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2)$ than $\mathbf{Q}_t (a_t \mathbf{I} + b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2)$.

We reiterate that in all our experiments, this instability can be avoided entirely by restarting correctly or using a safety factor of 1.05. Out of an abundance of caution, we rearrange Naive Gram Newton-Schulz (Algorithm 2) to avoid adding $a_t \mathbf{I}$ explicitly. We change

1. $\mathbf{Z}_t \leftarrow a_t \mathbf{I} + b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$ `// Apply  $h_t(\mathbf{R}_{t-1})$`
2. $\mathbf{Q}_t \leftarrow \mathbf{Q}_{t-1} \mathbf{Z}_t$
3. $\mathbf{R}_t \leftarrow \mathbf{Z}_t \mathbf{R}_{t-1} \mathbf{Z}_t$

to

1. $\mathbf{Z}_t \leftarrow b_t \mathbf{R}_{t-1} + c_t \mathbf{R}_{t-1}^2$
2. $\mathbf{Q}_t \leftarrow \mathbf{Q}_{t-1} \mathbf{Z}_t + a_t \mathbf{Q}_{t-1}$
3. $(\mathbf{RZ})_t \leftarrow \mathbf{R}_{t-1} \mathbf{Z}_t + a_t \mathbf{R}_{t-1}$
4. $\mathbf{R}_t \leftarrow \mathbf{Z}_t (\mathbf{RZ})_t + a_t (\mathbf{RZ})_t$

This change fixes all collected training examples in which symmetric GEMMs were less stable than non-symmetric GEMMs.

C Kernel Implementation Details

C.1 Symmetric GEMM Kernel Details

We implement all of our symmetric GEMM kernels with square cluster work tiles. Hopper uses cluster size (2,1) and thread block tile size (128,256), and Blackwell uses cluster size (2,1) and 2-CTA collaboration, in which the 2 thread blocks in the cluster collaborate on the same big (256,256) tile. Notably, highly optimized custom GEMM kernels on Hopper typically use Ping Pong Scheduling, in which the MMA of tile i and the epilogue of tile $i - 1$ are overlapped in two consumer warp groups.⁵ However, Ping Pong Scheduling uses more registers at once, and (128,256) is too large of a tile size for Ping Pong Scheduling, leading to register spillage. This is much slower than standard single producer warp, single consumer warp scheduling. Thus, our Hopper symmetric kernels do not use Ping Pong Scheduling. Blackwell GEMM kernels have no explicit conception of Ping Pong Scheduling, since by default in cuBLAS and most kernel libraries, two accumulators are kept in the new tensor memory hierarchy, and MMA is computed on one accumulator while the epilogue is computed on the other.

As a small implementation detail, note that the main diagonal of 256×256 cluster work tiles is part of the work assigned by the triangular scheduler. Since their transposed locations are identical to their current locations, we only write those values to general memory once—writing twice can cause inaccurate values or NaNs.

C.2 Implementation Strategy in Code

There are only two differences between the symmetric GEMM kernel and the standard GEMM kernel: the triangular scheduler and the transposed tile write in the epilogue. We design our symmetric kernel such that

⁵<https://pytorch.org/blog/cutlass-ping-pong-gemm-kernel/>

it is abstracted around the standard GEMM kernel to enable lightweight but maximally performant GEMM epilogue fusions. Using these abstractions, we are able to implement the symmetric GEMM kernel for both Hopper and Blackwell in just 160 lines, while achieving state-of-the-art performance.

We override the standard tile scheduler with our triangular scheduler and wrap the symmetric GEMM class around the GEMM with activation class. GEMM with activation itself is a wrapper around the Blackwell and Hopper default GEMMs. It supports writing two output tensors—the standard GEMM output (the preactivation) and the standard GEMM output with an activation function such as SwiGLU or ReLU applied (the postactivation). We define the activation function to be the identity and the postactivation tensor to be the inplace transpose of the preactivation tensor. Then, when the GEMM with activation class writes to the postactivation, it is really writing to the upper triangle with a transposed layout—this is exactly the intent of the symmetric GEMM kernel. We override the epilogue of GEMM with activation just to ensure we do not write twice to the diagonal tiles, for the correctness reasons mentioned previously.

C.3 Kernel Optimizations for Standard Newton-Schulz

Using just optimized CuteDSL kernels, we can accelerate standard Newton-Schulz with two changes.

Symmetric Matrix Multiplication As discussed above, the matrices $\mathbf{A} = \mathbf{X}\mathbf{X}^\top$ and $\mathbf{B} = b_t\mathbf{A} + c_t\mathbf{A}^2$ computed at each iteration of Newton-Schulz are symmetric by definition. Therefore, we use our symmetric GEMM kernels for these operations, reducing their FLOP cost by half.

Fused GEMM + Add The typical way to implement the non-symmetric multiplication $\mathbf{X} \leftarrow a_t\mathbf{X} + \mathbf{B}\mathbf{X}$ is to use `torch.baddbmm`, which calls cuBLAS under the hood. However, we offer a much faster implementation of this “Fused GEMM + Add” operation for Hopper. Unlike cuBLAS, our “Fused GEMM + Add” supports Ping Pong Scheduling for Hopper, which better hides the epilogue addition of $a_t\mathbf{X}$.

D Additional Experiments

D.1 Architecture and Optimization Setup

We now describe details of the experiments in [Section 8.1](#). (The benchmarks in [Section 8.2](#) are similar, but with minor architectural differences, synthetic data, and GH200s.)

Architecture. The models are decoder-only dense transformers (not mixtures of experts). They use grouped-query attention, sliding-window attention (window 2048), rotary position embeddings, and RMSNorm, at sequence length 8192 over a vocabulary of 100,352 tokens. Weights are stored and updated in MXFP8. [Table 4](#) lists the per-scale dimensions. Each run uses a single node of eight B200s.

Table 4: Model dimensions by scale. All share sequence length 8192, vocabulary 100,352, sliding window 2048, grouped-query attention, and rotary position embeddings.

Scale	Hidden dim	Layers	Heads	KV heads	MLP hidden dim
1B	1536	24	16	8	6656
3B	2304	32	18	6	9856
4B	2560	36	32	8	10752
7B	4096	32	32	8	11008
14B	5120	40	40	8	14592

Optimization. Matrix parameters are updated by the orthogonalizing optimizer under study—Muon, NorMuon, or their filtered variants. For matrix parameters, we multiply the base learning rate by $\sqrt{d_{\text{out}}/d_{\text{in}}}$. Embeddings, the language-model head, norm scales, and biases are updated by AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)

at the base learning rate, without the $\sqrt{d_{\text{out}}/d_{\text{in}}}$ scaling applied to the matrix parameters (the language-model head, in particular, also uses the base rate). We apply weight decay 0.01 to the matrix parameters only; the AdamW (non-matrix) groups use no weight decay. The sequence length is 8192. The learning-rate schedule is warmup–stable–cooldown: a 200-step linear warmup from 2×10^{-3} to the peak learning rate, a constant phase, and a cooldown over the final 25% of training back down to 2×10^{-3} . All reported held-out cross-entropies are taken at the end of cooldown.

D.2 Finer-Grained Timing Metrics

The main text reports GPU (device) time, the metric of interest for the orthogonalization compute. Here we report three per-step metrics side by side:

- **GPU (device) time**, read from CUDA events between two stream markers around `optimizer.step()`;
- **CPU (host) time**, from `time.perf_counter()` stopped immediately after `step()` returns and before any device sync — cost of kernel launch, Python, and any host-side syncs the optimizer performs;
- **Communication volume**, the number of bytes moved by the megabatch collectives (`all_to_all`, `all_gather`, `reduce_scatter`) at each step, measured by tallying the receive-side payload per-GPU.

As always, Newton-Schulz runs in half precision arithmetic. Results for a 14B model sharded over eight B200s and a 7B model on four GH200s appear in Table 5 and Table 6, respectively. Unlike in Section 8.2, we show all four combinations of kernel and Newton-Schulz variant. We see that Gram Newton-Schulz *without* symmetric kernels achieves up to a $1.2\times$ speedup. Otherwise, GPU timings are consistent with the results presented in Section 8.2. The NorMuon family has higher costs overall due to the overhead of its extra normalization steps. Our contributions still yield similar speedups: up to $1.5\times$ for symmetric kernels and Gram Newton-Schulz, $3.3\times$ for $f = 1/2$, and $5.6\times$ for $f = 1/4$. Table 5 shows that the CPU cost is negligible—just ~ 0.1 ms across configurations. This near-constant host cost is a direct consequence of CUDA-graph capture. Row selection, error feedback, and Gram Newton-Schulz issue many small operations and kernel launches. Without capture, the host would dispatch each one individually, and this per-launch overhead would dominate, especially for the filtered configurations, whose device work is small. Capture and replay instead collapse the entire step into a single graph launch, so the host issues one call regardless of how many kernels the step contains or how much they compute. The reported host time is therefore just the fixed cost of that one launch (~ 0.1 ms), essentially independent of the configuration. Finally, communication volume is directly proportional to the fraction f , as expected.

Table 5: Fine-grained per-step metrics for the distributed 14B / 8-GPU benchmark: GPU (device) time from CUDA events, CPU (host) time from `perf_counter` stopped before the device sync, and per-GPU communication volume of one step. We distinguish the GPU time between the Muon and NorMuon families of optimizers; CPU time and communication volume are reported once because neither depends on the family: host time is the fixed CUDA-graph launch cost, and communication volume is set by the parameter shapes and the fraction f .

Configuration	Kernels	Algorithm	NS frac.	GPU (ms)		CPU (ms)	Comm (MB)
				Muon	NorMuon		
Standard Newton-Schulz	PyTorch	Standard	1	153.8	174.1	0.1	5479
+ symmetric kernels	CuteDSL	Standard	1	132.7	153.4	0.1	5479
+ Gram NS	PyTorch	Gram	1	137.6	158.5	0.1	5479
+ kernels + Gram NS	CuteDSL	Gram	1	106.8	127.0	0.1	5479
+ filtering ($f = 1/2$)	CuteDSL	Gram	$1/2$	66.4	73.8	0.1	2740
+ filtering ($f = 1/4$)	CuteDSL	Gram	$1/4$	34.8	38.1	0.1	1370

D.3 Benchmarking all-to-all communications

In Section 7, we claim that reducing the number of communication rounds via megabatching is beneficial in two ways: (1) each round adds a fixed overhead not dependent on the size of the payload, and (2)

Table 6: Replication of Table 5 for a 7B-parameter model sharded over a node of four GH200s with NVLink.

Configuration	Kernels	Algorithm	NS frac.	GPU (ms)		Comm (MB)
				Muon	NorMuon	
Standard Newton-Schulz	PyTorch	Standard	1	360.6	385.4	6442
+ symmetric kernels	CuteDSL	Standard	1	274.4	315.0	6442
+ Gram NS	PyTorch	Gram	1	296.6	321.1	6442
+ kernels + Gram NS	CuteDSL	Gram	1	217.7	252.1	6442
+ filtering ($f = 1/2$)	CuteDSL	Gram	$1/2$	101.5	116.7	3221
+ filtering ($f = 1/4$)	CuteDSL	Gram	$1/4$	59.6	68.6	1610

small messages fail to saturate the interconnect. Here, we verify both claims directly with a standalone microbenchmark of NCCL `all_to_all` over NVLink, independent of any optimizer. On a single node of four H100s (SXM, NVLink-4 mesh, NCCL 2.29), we sweep the size of the payload per rank from 1 KiB to 1 GiB. We time each all-to-all with CUDA events (10 warmup, 50 timed iterations), taking the per-iteration time to be the maximum over ranks, and measure the bandwidth. We confirmed that the transport is a pure NVLink peer-to-peer communication, with no PCIe or network fallback and that the measured bandwidths agree with the `nccl-tests alltoall_perf` tool to within 1%.

Figure 21 shows the results. Both effects discussed in Section 7 are clearly visible. The latency floor is $\sim 25 \mu\text{s}$ (left panel), of which $\sim 15\text{--}17 \mu\text{s}$ is host-side dispatch (as measured separately with `perf_counter` and no device synchronization). Without megabatching, we pay this cost over and over. Bandwidth climbs by more than an order of magnitude as the per-link payload grows (right panel), so coalescing many small all-to-alls into one megabatch increases the effective bandwidth of each transfer.

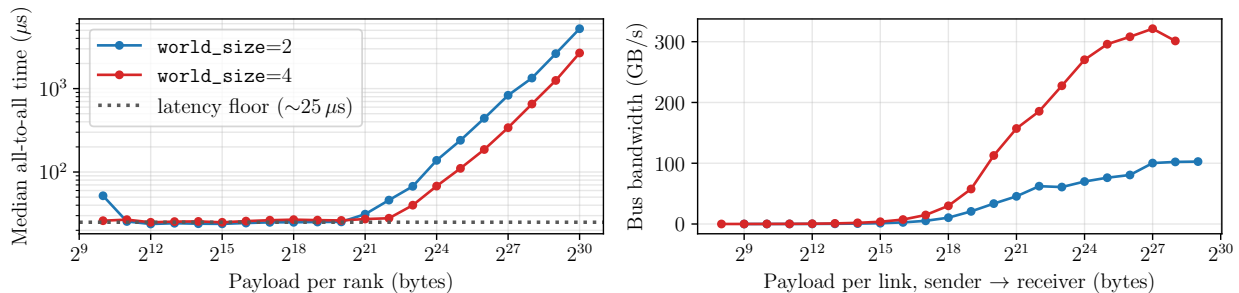


Figure 21: NCCL `all_to_all` over NVLink on $4 \times \text{H100}$. **Left:** median all-to-all time versus payload per rank (sender) over 50 trials. For small payloads, runtime is dominated by a fixed latency. **Right:** bus bandwidth versus payload per link (sender $\rightarrow$ receiver). Bandwidth is near zero for small messages and does not reach 80–90% of its peak until the per-link payload is $\sim 16\text{--}32$ MiB. At `world_size = 4`, a 256 KiB per-link payload achieves only $\sim 10\%$ of the peak bandwidth.

D.4 Ablations

Tuning the baseline The experiments in Section 8.1 compare Dion3 to a NorMuon baseline. Table 7, below, demonstrates that this baseline is properly tuned for a fair comparison.

Dion3 with no compression As a control, we run Dion3 with $f = 1$, which should reduce to plain NorMuon. Figure 22 confirms this at 1B; the two curves coincide throughout training, and their final validation losses differ by 0.0005.

Timings with NorMuon family Section 8.2 compares the runtime of Muon to the Muon version of Dion3. Here, in Figure 23, we repeat this benchmark with the NorMuon family. Results are quite alike, but

Table 7: Tuning hyperparameters (learning rate η , momentum μ) of NorMuon at 1B. Final validation loss on 100B tokens of ClimbMix. Winner: $\eta = 0.01$, $\mu = 0.9$ (bold).

Sweep η (at $\mu = 0.9$)		Sweep μ (at $\eta = 0.01$)	
η	CE	μ	CE
0.005	2.220	0.9	2.194
0.01	2.194	0.949	2.201
0.02	2.215	0.974	2.217
0.03	2.237	0.987	2.238

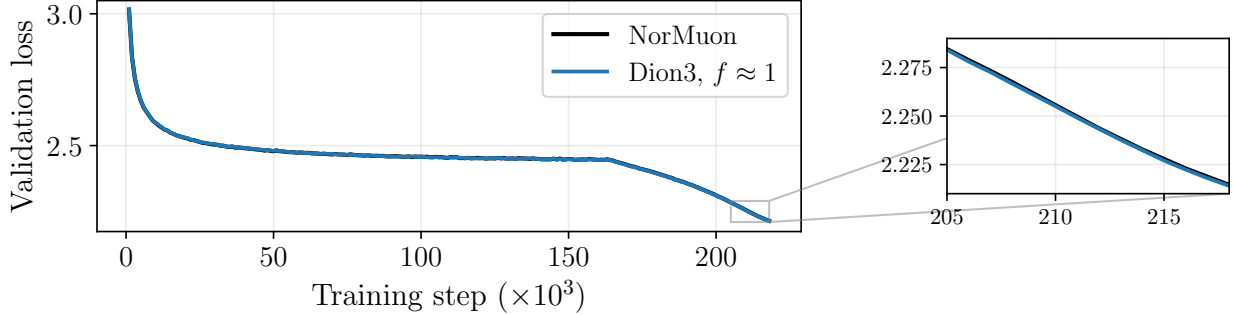


Figure 22: Validation loss at 1B for NorMuon and for Dion3 at $f \approx 1$, on 100B tokens of ClimbMix. As expected, Dion3 tracks NorMuon throughout, ending at 2.2141 versus 2.2146.

due to the extra overhead of NorMuon’s normalization steps, it benefits slightly less from our contributions, which target the orthogonalization step.

E Case Studies of End-to-End Training Time

The share of end-to-end training time taken up by Newton-Schulz can vary widely depending on the training setup. To explain this variability, we analyze two idealized scenarios. In one, standard Newton-Schulz takes 2% of training time; in the other it takes 17%.

Case Study 1: Standard Newton-Schulz takes 2% of Kimi K2 training time

The following analysis gives a very optimistic estimate of the optimizer’s wall clock time. We assume an efficient training infrastructure with highly optimized pipeline parallelism. Moreover, we assume that the optimizer step of each pipeline stage is completely hidden behind the backward pass of the next pipeline stage.

Kimi K2 Thinking is a 1.1 trillion parameter model with 32 billion active parameters. It has 1 dense layer followed by 60 MoE layers [21]. It is pretrained with 256-GPU model parallel groups, 16-way pipeline parallelism, 16-way expert parallelism within each pipeline stage, and a huge batch size of 67 million tokens.

We use a single H100 to approximate the share of each training step’s runtime occupied by Newton-Schulz in this setting under the following assumptions:

1. The training cluster consists of 256 nodes of eight H100s each (2048 GPUs in total), connected with NDR 400 Gb/s InfiniBand inter-node (8 NICs per node, 1:1 NIC-to-GPU ratio) and NVLink 4.0 intra-node. This is the size of the cluster used to train DeepSeek-V3, with upgraded hardware [9].
2. Training in `bfloat16` attains 40% model flop utilization (MFU), which is typical for MoEs at this scale on H100s.

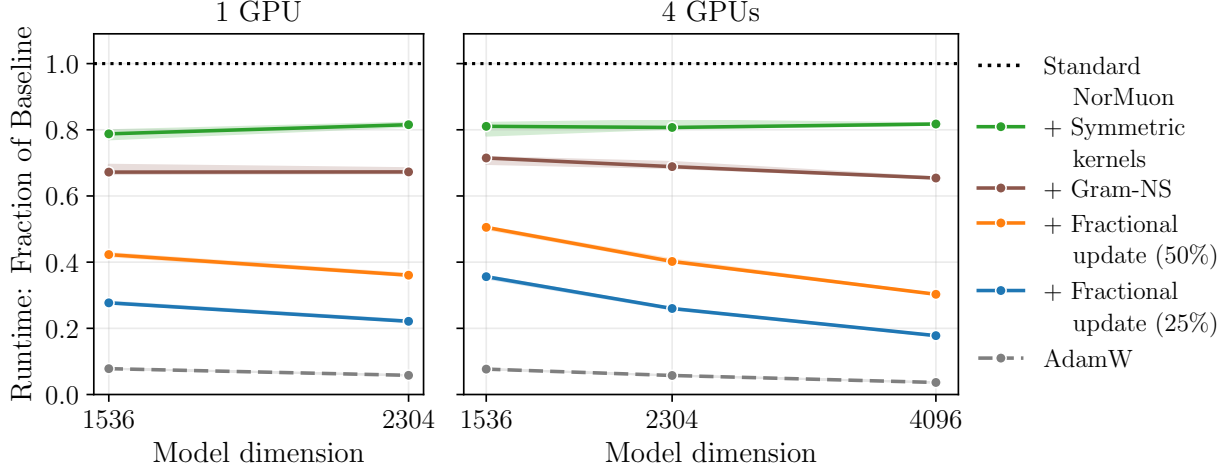


Figure 23: Optimizer step time (excluding forward/backward pass) relative to standard Muon across model scales when training on 1 GH200 (left) and FSDP over 4 GH200s (right). Each colored line adds one of our contributions on top of the previous one; AdamW is shown for reference. Lines show median over 25 steps and bands show interquartile range. Compare to Figure 6.

3. The only non-overlapped optimizer time is that of the last pipeline stage to complete its backward pass (i.e., pipeline stage 1 of 16). The optimizer steps of pipeline stages 2 to 16 are fully hidden behind the backwards of stages 1 to 15.
4. Pipeline stage 1 contains the dense layer and 3 MoE layers.

Under these assumptions, the optimal way to partition the Newton-Schulz work of pipeline stage 1 is as follows. Each of the 16 GPUs in pipeline stage 1’s expert parallel group gets

$$\frac{384 \text{ experts/layer} \times 3 \text{ MoE layers}}{16 \text{ GPUs}} = 72 \text{ experts/GPU} = 216 \text{ expert up-gate-down/GPU}.$$

Each of the 16 GPUs has its own unique expert weights, so no communication is needed. Pipeline stage 1 also contains the dense MLP’s three 7168×18432 weights (up/gate/down) and three shared experts. Orthogonalizing the dense MLP weights is the dominant cost, so they are sent to three different GPUs; the shared experts are split amongst the remaining 13 GPUs. Thus, the orthogonalization time for pipeline stage 1 is the time it takes for a single GPU to orthogonalize 216 expert up/gate/down weights and 1 dense up/gate/down weight. As benchmarked in Figure 10, standard Newton-Schulz implemented in PyTorch takes 315 ms to do this. Per our assumption, stage 1’s orthogonalization step is the only one that is not overlapped.

Having estimated the Newton-Schulz time, we now estimate the end-to-end wall clock time of an entire Kimi K2 global training step. We use the standard estimate of $6NB$ FLOPs for the forward and backward pass, where N is the number of active parameters and B is the global batch size. Given

- Active parameters: $N = 32 \cdot 10^9$
- H100 peak: $P = 989 \cdot 10^{12}$ FLOP/s
- Model flop utilization: $\text{MFU} = 40\%$
- Cluster size: $G = 2048$ GPUs
- Global batch size: $B = 67 \cdot 10^6$ tokens

we have

$$\text{sec/batch} = \frac{6N \times B}{P \times \text{MFU} \times G} = 15.9$$

Thus, Newton-Schulz takes approximately $\frac{315 \text{ ms}}{15900 \text{ ms} + 315 \text{ ms}} = 1.9\%$ of total pretraining wall clock time in this setting.

Case Study 2: Standard Newton-Schulz takes 17% of Llama3-70B SFT time

Llama3-70B is an 80-layer dense model with hidden size 8192, intermediate size 28672, and grouped query attention with $\mathbf{W}_K, \mathbf{W}_V$ of size 1024×8192 and $\mathbf{W}_Q, \mathbf{W}_O$ of size 8192×8192 [14]. Supervised finetuning (SFT) typically uses small batch sizes, ranging from 32 to 256 sequences [9].

We analyze the following SFT case:

1. Training uses 32 H100s across 4 nodes (8 GPUs per node).
2. Training in `bfloat16` hits 40% MFU.
3. Weights are sharded evenly across GPUs using FSDP, and the exposed Newton-Schulz time is that of 80 layers/32 GPUs ≈ 3 layers. Each layer has 3 MLP weights (up, gate, down), and the attention weights $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, and $\mathbf{W}_O$.

According to our benchmarking, standard Newton-Schulz of

- nine 8192×28672 weights takes 739 ms,
- six 8192×8192 weights takes 156 ms,
- six 1024×8192 weights takes 2.32 ms,

totaling 897 ms. Given

- Parameters: $N = 70 \cdot 10^9$
- H100 peak: $P = 989 \cdot 10^{12}$ FLOP/s
- Model flop utilization: $\text{MFU} = 40\%$
- Cluster size: $G = 32$ GPUs
- Global batch size: $B = 64 \text{ sequences} \times 2048 \text{ tokens/sequence} = 131,072 \text{ tokens}$

we have

$$\text{sec/batch} = \frac{6N \times B}{P \times \text{MFU} \times G} = 4.35 \text{ s}$$

Thus, Newton-Schulz takes approximately $\frac{897 \text{ ms}}{4350 \text{ ms} + 897 \text{ ms}} = 17\%$ of total SFT wall clock time in this setting.